Parallel ensemble data assimilation in ICESEE
Algorithms, model coupling and reproducible ice-sheet simulations

Brian Kyanjo

Technical report and preprint draft
10 November 2026

Abstract

Ensemble data assimilation combines uncertain model trajectories with observations, but its application to ice-sheet models requires us to coordinate several computational scales. Ensemble members evolve independently during forecasts, interact through an analysis, and may themselves require spatially distributed solvers. We describe developments in the Ice Sheet State and Parameter Estimator (ICESEE) that connect these stages through explicit state ownership, ensemble-space analysis, model adapters, and recoverable output. We explain four execution modes: serial orchestration, partially parallel forecasts and row updates, file-backed ensemble analysis, and application-specific native execution. We derive the implemented factor transform and its streamed products, distinguish numerical filtering choices from process scheduling, and describe the treatment of initialization, observation errors, process noise, and state–parameter estimation. We describe hybrid basal-friction inversion through its objective, regularization, activation clock, and state–parameter handoff, and distinguish it from optional apparent-mass-balance and bed-increment inference. A common state-column clock aligns initial conditions, forecasts, observations, and restart progress. Retained verification includes a five-year, 32-member Icepack calculation in three modes, a short restart across analysis events, and real-MPI integration tests. All three five-year simulation steps exited successfully; subsequent verification was completed separately after allocation timeouts during output reads. A further two-rank comparison with final-time observations confirms complete histories and Mode-1/Mode-2 agreement for a small fixture. A new coarse, data-independent Icepack exercise completes all four modes with closed-output integrity checks. These results establish the tested numerical and operational behavior. Controlled scaling and independent physical-skill experiments remain separate questions. The accompanying source maps and student procedures provide the detail needed to reproduce and extend the implementation.

Implementation basis. The numerical source is ICESEE `features_debug`, commit `3748e416d3a8120cc860119bd7224c6f9ad41ff7`. We identify this development snapshot by its commit rather than assigning a new release number. Source and deployment provenance are detailed in Appendix A.

Contents

1 Introduction **3**

2 Related work and the scope of the developments **4**

3 Ensemble formulation and uncertainty treatment **5**

4 Model coupling and physical state representation **8**

5 Hybrid inversion and parameter inference **11**

6 Physical inference and supporting developments **15**

7 State ownership and process topology **17**

8 Execution modes and parallel work distribution **18**

9 Computational cost, memory, and history storage **27**

10 The state-column time contract **29**

11 Checkpoints, history and failure recovery **30**

12 Verification design **31**

13 Results from retained validation records **31**

14 Reproducible sample applications **33**

15 Discussion and directions for scientific evaluation **36**

16 Conclusions **37**

A Source identity and the computational experiment **38**

B Installation and environment operation **39**

C A production operations protocol **40**

D Source map for a new developer **41**

E Evidence inventory **42**

F The minimum run record **42**

G Failure diagnosis and decisions **43**

H A worked first-run procedure **43**

I A student learning sequence **44**

1 Introduction

Ice-sheet observations describe only part of a dynamically evolving system. Surface elevation and velocity are increasingly accessible, whereas basal conditions, internal state, and some forcing fields remain weakly constrained. We therefore seek estimates that combine observations with a model of ice flow while representing uncertainty in both the initial state and the quantities that control subsequent evolution. Data assimilation provides this connection by updating a model trajectory as observations become available. Its practical formulation depends on how uncertainty is represented, which variables are observed, and how the observation information propagates through the model (Carrassi et al., 2018).

An ensemble approach represents that uncertainty through multiple model realizations. During a forecast, each member evolves under the model and its assigned uncertain initial conditions or forcing. During analysis, differences among the members provide relationships between observed and unobserved quantities. For an ice-sheet calculation, this interaction can involve geometry, diagnostic velocity, surface mass balance, basal melt, or a parameter controlling the stress balance. State correction and parameter inference are related but have different physical meanings: a velocity observation can constrain an uncertain parameter through ensemble cross covariance or through a separate inverse problem.

These statistical operations also define a computational problem. Members can be forecast independently, but each analysis combines information across the ensemble. A large member may require several MPI ranks for the model solve, while a long experiment requires many saved ensemble states. Consequently, a method that is inexpensive in ensemble space can still be limited by the model's state representation, a root process holding the forecast ensemble, or the volume and organization of output. We distinguish these limits because they lead to different parallelization strategies. Increasing the number of forecast workers does not necessarily reduce analysis memory, and reducing retained history does not reduce the size of one physical member.

We developed ICESEE to connect ensemble filtering to several model environments through model-specific forecast and inversion hooks. The earlier ICESEE model-description paper introduced the framework, filtering alternatives, and applications with ISSM and Icepack (Kyanjo et al., 2026). In the present report, we describe later developments in the numerical implementation and its execution paths. We focus on the relationship between ensemble algebra, model-owned state, physical time, and recoverable output. This relationship is important when experiments are operated by different students or continued after an interruption, since numerical reproducibility depends on more than the update equation alone.

We organize the developments around four execution modes. Mode 0 provides a serial lifecycle and serial method alternatives. Mode 1 distributes forecasts over model groups and applies a root-computed analysis transform to distributed state rows. Mode 2 stores ensemble states in files and streams observation and state rows through an ensemble-space analysis. Mode 3 selects an application-specific native lifecycle, including genuinely spatial model fields where the adapter exposes them. We explain initialization, scheduling, communication, memory, and output for each mode, rather than treating the mode number as a measure of scalability.

Our numerical description retains the exact factor construction in the inspected source. In particular, we distinguish its finite-sample observation factor from a textbook gain expressed with an arbitrary prescribed covariance. We also describe replicated versus distributed state, a shared state-column time convention, deterministic stochastic identities, and checkpoint progress. These developments allow us to state and test which operations should remain equivalent when ownership or storage changes.

We evaluate the implementation using retained Icepack simulations, restart and output checks, continuous integration, and additional small MPI comparisons. The reported measurements establish the tested configurations and their numerical integrity. They motivate, but do not replace, controlled scaling or physical-skill studies. The detailed appendices and companion

handbook provide a second reading route: students can follow a complete small case, locate each implementation boundary, and use the same evidence conventions when extending the framework.

2 Related work and the scope of the developments

2.1 Ensemble filtering and numerical representations

The Ensemble Kalman Filter uses an ensemble to propagate forecast uncertainty through a nonlinear model and to construct an approximate analysis. The practical formulation described by Evensen (2003) connects the ensemble representation to numerical operations suitable for large models. The ensemble can provide covariance information without requiring a stored state-by-state covariance in every formulation. This is the relevant starting point for our distinction between the statistical update and the arrays used to evaluate it. File-backed storage and ensemble representations have substantial precedents; our contribution concerns their integration with ICESEE's model, ownership, and time conventions.

Observation treatment distinguishes stochastic and deterministic ensemble filters. Burgers et al. (1998) describe the role of observation perturbations in a stochastic analysis. Deterministic square-root approaches instead construct an analyzed ensemble without an independently perturbed observation for every member, with several possible transforms satisfying the desired covariance relationship (Tippett et al., 2003). The deterministic EnKF of Sakov and Oke (2008) provides a related approximation through distinct mean and anomaly updates. These alternatives motivate the serial method interface in ICESEE. They also show why an execution mode or a matrix called a transform does not, by itself, identify a particular filter.

Local analysis provides another connection between statistical structure and computation. Hunt et al. (2007) develop a local ensemble transform Kalman filter for spatiotemporal systems. Their LETKF is a particular deterministic local filtering formulation. In contrast, the grouped local operations described here retain ICESEE's selected stochastic factor and group targets with identical observation support. Grouping can reuse a factorization, and distributed ownership can change where it is computed, without making the resulting algorithm LETKF. Similarly, the correlation functions of Gaspari and Cohn (1999) provide widely used compact-support constructions, but neither the function family nor a matrix-free representation determines an appropriate physical localization radius for an ice-sheet observation network.

2.2 Data assimilation for ice-sheet states and parameters

Ice-sheet ensemble studies demonstrate how observations constrain both transient states and uncertain model properties. Bonan et al. (2014) use an ETKF approach for initial-state and parameter estimation in an idealized flowline experiment. That work links transient state estimation to parameter uncertainty in a controlled setting, providing a useful model for the distinction between prior specification and subsequent analysis. Its filter and shallow-ice experiment are separate from the native Icepack implementation evaluated in the present report.

Gillet-Chaulet (2020) assimilate surface observations in a transient marine ice-sheet model and examine their ability to constrain state and basal conditions in synthetic experiments. Choi et al. (2025) use ensemble filtering and observing-system simulation experiments to investigate state and parameter estimation, including sensitivity to the observation network and uncertainty choices. Together these studies motivate our attention to observation support, joint state–parameter relationships, and physical consistency after analysis. They do not imply that a particular new network, parameter prior, or execution path inherits the same skill. We therefore separate tests of computational equivalence from evaluation against known truth or withheld observations.

2.3 Model software and the implementation boundary

ICESEE connects to numerical models with different internal representations. Icepack supplies glacier-flow models built on variational finite-element formulations (Shapero et al., 2021); its Firedrake foundation provides abstractions for meshes, functions, and generated finite-element operations (Rathgeber et al., 2016). ISSM combines ice-sheet models, parallel numerical solvers, and inverse methods in a different software environment (Larour et al., 2012). PETSc supplies distributed numerical objects and solver infrastructure in relevant backends (Balay et al., 2025). These capabilities motivate a native field interface, but the backend's internal parallelism does not establish what its Python adapter can transfer. We document the actual boundary independently for each application.

The earlier ICESEE manuscript and its public author replies already discuss observation-error modes, covariance localization, and grouped local analysis (Kyanjo et al., 2026,b). We retain those elements as necessary numerical context. The later developments examined here concern explicit replicated/distributed ownership, file-backed row execution, persistent native members, state-column timing, and reproducible recovery and verification. We make the scope concrete through the selected source revision and application-specific evidence rather than identifying all generic ensemble or parallel-computing ideas as new.

3 Ensemble formulation and uncertainty treatment

3.1 State and observation notation

Let n be the number of packed state entries, m the active observation count and N_{e} the ensemble size. We arrange members in columns,

$$X^f = [x_1^f, \ldots, x_{N_{\mathrm{e}}}^f] \in \mathbb{R}^{n \times N_{\mathrm{e}}}, \qquad A = X^f - \bar{x}^f \mathbf{1}^T. \tag{1}$$

For a potentially nonlinear observation operator h, evaluate each forecast member first:

$$Y = [h(x_1^f), \ldots, h(x_{N_{\mathrm{e}}}^f)], \quad \bar{y}^f = Y\mathbf{1}/N_{\mathrm{e}}, \quad Y' = Y - \bar{y}^f \mathbf{1}^T. \tag{2}$$

In particular, $h(A)$ is not a replacement for Y' when h is nonlinear. The observation vector y has length m. The conventional stochastic analysis provides the familiar reference expression

$$X^a = X^f + AY'^T\{Y'Y'^T + (N_{\mathrm{e}} - 1)R\}^{-1}D, \qquad D = y\mathbf{1}^T + E - Y, \tag{3}$$

with suitable assumptions on errors and the inverse (Evensen, 2003; Burgers et al., 1998). This equation establishes dimensions and terminology. The implemented factor formulation below must be stated separately.

3.2 Prior members, forecast uncertainty, and stochastic identity

We write the forecast and analysis as separate stages,

$$x_{i,k+1}^f = \mathcal{M}_{k \to k+1}(x_{i,k}^a; \theta_i) + \delta q_{i,k}, \qquad x_{i,k+1}^a = \mathcal{A}_{k+1}(X_{k+1}^f, y_{k+1})_i, \tag{4}$$

where $\mathcal{M}$ is supplied by the model adapter, θ_i denotes uncertain member parameters, and δq is a configured forecast increment. This notation describes the orchestration boundary; a physical adapter can enforce constraints or obtain diagnostic quantities through its own nonlinear solve.

The initial ensemble is a realized prior, not only an ensemble size and a standard-deviation vector. Generic initialization combines the model's member construction with configured perturbations and optional spread adjustment. We can write $x_{i,0} = x_{i,0}^{\mathrm{model}} + \delta x_{i,0}$, while retaining the selected variables, generator, spatial scales, and seeds as part of that definition. A native application can instead construct persistent fields directly and need not perturb the same variables.

In the retained PIG comparison, native initial thickness and surface are unperturbed relative to the reference, while generic initialization uses its configured perturbation policy. This difference is important when interpreting comparisons among execution modes.

The canonical forecast-noise routine maintains a separate autoregressive state for each member and state-variable block. At a cycle where noise is scheduled, its update has the form

$$q_{i,k}^{(v)} = \alpha q_{i,\text{previous}}^{(v)} + \sqrt{1-\alpha^2}\,\xi_{i,k}^{(v)}, \tag{5}$$

$$\delta q_{i,k}^{(v)} = \sqrt{\Delta t}\,\sigma_Q^{(v)} \rho\, q_{i,k}^{(v)}, \tag{6}$$

with the selected spatial field ξ, variable amplitude σ_Q, and code-defined normalization ρ. In the inspected drivers, $\alpha = 1 - \Delta t/\tau$ is used with a bounded fallback when necessary; the normalization also depends on the configured horizon. Equation 6 states the implemented recurrence without assuming that every field generator produces the same physical covariance. Noise can be scheduled at observation-ending forecasts, every forecast, or disabled. When events are intermittent, the retained stochastic state advances according to that policy, so timing is part of the noise definition.

Member, cycle, variable, and base-seed identifiers determine the canonical streams rather than MPI rank or round order. Initialization and forecast streams use separate namespaces. The graph field option can additionally key white-noise values by physical coordinates so a change in mesh numbering does not change their spatial identity. The native PIG bridge reuses the canonical forecast generator and scatters increments into owned rows; a separate helper with a native seed-related name is not necessarily the active policy. On restart, deterministic replay reconstructs the necessary autoregressive history without advancing the restored physical fields. We retain a policy fingerprint so this replay refers to the same time grid, amplitudes, coordinates, and schedule.

3.3 Observation factors and parameter estimation

For the explicit Gaussian factor, the implementation draws an observation-by-member matrix scaled by the active standard deviations and centers each row across members. If Ξ contains the generated standard normal values and $J = I - \mathbf{1}\mathbf{1}^T/N_\text{e}$, we can express this factor as

$$E = \operatorname{diag}(\sigma_1, \ldots, \sigma_m)\Xi J. \tag{7}$$

Its finite-ensemble sample covariance is a realized quantity. The spatially generated option replaces the independent row draws by configured fields before selection and centering. The legacy option sets $E = Y'$ and uses unperturbed innovations. These choices change the analysis factor and are therefore scientific configuration, rather than interchangeable storage options. Active observation standard deviations are checked for finite positive values; an object named `R` in an output file must be interpreted through its producer and consumer, since the observation pipeline can store standard deviations under that name.

We distinguish joint estimation from hybrid inversion. In joint estimation, the packed member contains both state and parameter blocks,

$$z_i = \begin{bmatrix} x_i \\ \theta_i \end{bmatrix}, \qquad P_{z,y}^f \approx \frac{1}{N_\text{e}-1} Z' Y'^T, \tag{8}$$

where Z' contains the augmented ensemble anomalies, so cross relationships can update unobserved parameters. Their forecast evolution depends on the adapter and declared uncertainty policy. In a hybrid workflow, an analyzed state is passed to a model-specific inverse solve and the resulting parameter is restored through that model interface. These are different estimators, even when both are described as basal-friction inference. Optional SMB/bed regularization, domain gates, delayed activation, blending, and physical projection add further prior assumptions. We retain their settings and ordering when testing equivalence between execution paths; physical identifiability is evaluated through a separate observation experiment.

3.4 The implemented factor and ensemble transform

`StochasticAnalysisProducts.add_chunk` in `src/parallelization/distributed_analysis.py` forms a factor $F = Y' + E$. For `stochastic_R` and `generated_R`, an explicit error array E, aligned with the forecast observation rows and members, is required. The innovations are $D = y\mathbf{1}^T + E - Y$. For `legacy_prior_anomalies`, the implementation instead sets $E = Y'$ and $D = y\mathbf{1}^T - Y$. These policies are not interchangeable descriptions of an identical experiment.

For observation chunks indexed by b, accumulate

$$C = \sum_b Y_b'^T F_b, \qquad G = \sum_b F_b^T F_b, \qquad B = \sum_b F_b^T D_b. \tag{9}$$

Each product has shape $N_\mathrm{e} \times N_\mathrm{e}$. After reduction over distinct observation owners, `ensemble_transform_from_products` symmetrizes G, computes its eigendecomposition, orders eigenvalues from largest to smallest, clips negative values to zero, and retains enough eigenvalues to reach a default fraction 0.999 of their sum. An additional machine-precision threshold excludes numerically negligible retained values. With the resulting truncated pseudoinverse G_q^+, the returned transform is

$$T = I + CG_q^+G_q^+B, \qquad X^a = X^f T. \tag{10}$$

The two pseudoinverse factors follow the factor-space identity below and are retained in the streamed computation.

To interpret the products, for an untruncated singular value decomposition $F = U\Sigma V^T$,

$$(FF^T)^+ = F(F^TF)^+(F^TF)^+F^T. \tag{11}$$

Consequently the product form represents $Y'^T(FF^T)^+D$ without an observation-by-observation matrix. With truncation, the corresponding retained singular subspace must be used consistently. Since $\mathbf{1}^TY'^T = 0$, $\mathbf{1}^T(T - I) = 0$ in exact arithmetic, so applying the increment through raw members or anomalies is consistent in that algebraic sense.

This factor form does not by itself prove equality to Eq. 3 for an arbitrary supplied R. In a finite sample,

$$FF^T = Y'Y'^T + EE^T + Y'E^T + EY'^T. \tag{12}$$

The cross terms do not vanish merely because two random variables are independent in expectation. The retained spectrum, error generator, centering and scaling are all part of the numerical method. We therefore retain these choices when testing numerical equivalence, and assess any intended relation to another covariance formulation separately. The present developments preserve the selected implementation while changing ownership and storage.

3.5 Inflation, localization and physical postprocessing

The factor-form representation avoids explicit allocation of the state covariance $AA^T/(N_\mathrm{e} - 1)$. It does not remove finite-ensemble sampling error or justify ignoring localization. The code includes global and grouped local analysis paths, with statistical motivation distinct from the storage strategy (Gaspari and Cohn, 1999; Hunt et al., 2007). A grouped local request describes target state rows, observation coordinates, observation kinds and permitted couplings. All accepted observation kinds for one target enter the same assembled local transform; treating each kind as a separate sequential filter can change the result.

Inflation changes spread and therefore belongs to the experiment, even when it is applied in a bounded row loop. We retain its configured value together with the selected analysis path. The default PIG file inspected here specifies 1.15; that is an example value, not a recommendation for every model. Localization length scales and support rules require a scientific justification for each observation network.

A model adapter can supply a post-analysis consistency operation, whose effect depends on the application. The current PIG native callback synchronizes ghosts after owned values are restored; surface, flotation, friction, and diagnostic velocity are recomputed during a subsequent forecast, rather than guaranteed immediately after the EnKF update. We therefore distinguish the analyzed snapshot from a fully rediagnosed physical state. A separately selected projection can also alter the nominal analysis increment. The projection policy and its constraints are included in the experiment definition.

The shared mode-1/mode-2 control helper first relaxes increments, optionally caps them by variable, inflates anomalies about the controlled row mean, restores frozen variables and applies the bed-update gate. Model geometry and inference callbacks follow in their configured paths. These operations can change the posterior. The streamed spectral-energy setting is not a universal switch: some direct/local paths retain a hard-coded 0.999 threshold. A control comparison therefore refers to the active implementation, including this distinction.

The code detects ensembles whose observed-space anomalies are negligible compared with the forecast magnitude. In the distributed product implementation the default relative threshold is 10^{-8}. Refusing that analysis prevents division by round-off directions. The realized initial ensemble and configured process-noise schedule supply the spread entering this check. An added perturbation or changed threshold would define a different experiment.

4 Model coupling and physical state representation

4.1 The forecast and observation boundary

We separate shared ensemble orchestration from model-specific evolution. An application supplies initialization, forecast, observation, consistency, and, when used, inversion operations. The generic interface resolves a hook module for the selected example and passes a flat runtime context containing the scientific settings and required model objects. Shared code controls member identity and assimilation timing, while the adapter controls how its fields are represented and how one model interval is advanced.

For an ice-sheet geometry, this boundary connects a prognostic mass balance to diagnostic flow and geometric constraints. Using a positive basal-loss convention for illustration, the thickness balance has the form

$$\frac{\partial h}{\partial t} + \nabla \cdot (h\mathbf{u}) = a_s - a_b, \tag{13}$$

where h is thickness, $\mathbf{u}$ is depth-averaged velocity, and a_s, a_b describe surface gain and basal loss. The selected model defines the actual discretization, sign conventions, stress balance, and boundary conditions. An analysis that changes thickness, surface, or a basal coefficient can therefore require a new diagnostic solve or geometric projection. Applications differ in when they perform it: a supplied post-analysis callback can provide the step, whereas current native PIG synchronizes ghosts after analysis and recomputes the relevant diagnostics during a subsequent forecast. This timing is part of the numerical calculation.

Icepack exposes finite-element fields through Firedrake, allowing the native adapter to retain functions and read/write owned degrees of freedom (Shapero et al., 2021; Rathgeber et al., 2016). The registry describes field order, owned ranges, and synchronization rather than replacing the model's solver. In the PIG native path, the same member context persists through successive calls. PETSc/Firedrake communicator lifetime and owned/ghost semantics remain relevant to this implementation even though the algebraic analysis uses compact NumPy blocks (Balay et al., 2025).

4.2 Different models expose different capabilities

ISSM's coupled Python/MATLAB boundary transfers complete members even though the internal model can use parallel finite-element solves and inverse methods (Larour et al., 2012). Its ICESEE native registration can schedule members in rounds and reuse an external worker, but that does not expose spatially partitioned state to the shared analysis. We describe it as a whole-member boundary and retain the distinction between ICESEE worker ranks and internal solver processes.

The flowline application provides a small state with its own model equations and initialization, whereas the lightweight Lorenz application supplies an inexpensive nonlinear integration case. The current `lorenz96` directory contains the three-variable Lorenz-63 equations; we retain the executable path and identify the equations explicitly below. These reduced models are useful for isolating driver and communication behavior before evaluating a finite-element glacier workload.

Native registration is scoped by model and example. A shared policy can require one spatial rank, while a specialized PIG policy adds spatial groups and restart. An optional observation/inversion/local-analysis capability is used only through a binding that implements it. This permits continued use of the established generic hooks while developing a richer field interface, without claiming that every model already has the same native representation. The source map in the appendices identifies the relevant modules and protocols.

4.3 Coupling Icepack to the generic and native lifecycles

For a generic Icepack example, the application first constructs its mesh, function spaces, physical coefficients, initial geometry, and solver context. Its ordered `vec_inputs` then defines how fields such as thickness, velocity components, surface, and selected uncertain parameters cross the ICESEE boundary. The initialization hook returns member values in that layout. The forecast hook reconstructs or accesses the required functions, advances the model for one interval, and returns the fields expected by the shared driver. Observation IDs and physical coordinates connect the observation registry to these packed variables. A consistent dimension alone is insufficient if the packed order or mesh numbering has changed.

The native PIG adapter retains Firedrake member functions rather than treating a full NumPy vector as the model's primary state. Its field registry exposes owned entries for analysis and checkpoints, and restoring an analysis updates those same functions before the application callback. In the current PIG binding that callback synchronizes ghosts; surface, flotation, friction, and diagnostic velocity are recomputed in a subsequent forecast rather than all being rediagnosed immediately after EnKF analysis. Each member's model context uses the spatial communicator assigned to its ensemble group. The native process-noise wrapper routes a canonical increment to owned rows through a coordinate-derived map, maintaining a distinction between stochastic-workspace memory and physical field ownership. This allows the model solve and the ensemble analysis to share the same physical state without a complete physical-member gather at their interface.

In this PIG configuration, thickness is prognostic, velocity is diagnostic, and surface follows geometric relations. The packed melt slot is also not sufficient evidence of a persistent estimated forcing: the native forecast reconstructs basal melt from the configured experiment before advancing the model. We therefore distinguish values stored in an analyzed snapshot from the forcing actually consumed by the next forecast.

Icepack applications nevertheless use different bindings. The synthetic-stream example currently uses a whole-member adapter with one spatial rank and a file-backed native analysis policy. PIG uses a specialized spatial/global-analysis runner and its own validated checkpoint/resume policy. Available local-analysis infrastructure is an implementation component, not evidence that every Icepack runner exposes every localization or inference setting. We therefore identify the example, mode, adapter, and analysis policy together when defining a coupled experiment.

4.4 Coupling ISSM through an external model process

ISSM presents a different boundary. The application prepares model and member files and communicates with a persistent MATLAB server. A forecast or inversion request invokes the model-specific MATLAB/ISSM workflow and returns a complete member's fields to Python. The external solve can use several internal processes, controlled by its own resources and `model_nprocs`, while the ICESEE worker remains a separate process. The active bridge exchanges commands and completion status through per-worker command/status files, using absolute paths and atomic running/done/error state; it is a file-IPC boundary rather than an assumed TCP-port interface. These internal ranks are not interchangeable with ranks in ICESEE's world or with a native spatial field partition.

The member ID must remain explicit in this workflow. It determines member directories, parameter files, returned fields, and server requests; it cannot be replaced by world rank when a worker advances more than one member. Likewise, setup routines that contain world collectives are entered once by the intended participants, rather than once for every independently scheduled server request. The `ISMIP_Choi` native runner uses round-assigned complete members and member-wise inversion while preserving this external process boundary. Its concurrency and resident-worker properties differ from a strict spatially distributed-state invariant. The current whole-member adapter has no reactivation callback, so the default native pool retains all locally assigned member arrays; a bounded number of MATLAB workers does not bound the resident member-array count. The Choi runner writes checkpoint shards, but it does not wire a validated checkpoint load/resume lifecycle. These write and worker capabilities are reported separately from restart or streaming support.

Field names and block order are example-specific. The Choi application uses six nodal blocks, `Thickness`, `Surface`, `Vx`, `Vy`, `bed`, and `coefficient`. The basal-friction-variation example instead uses `Bed` and `FrictionCoefficient` for the last two names, and keeps volume diagnostics outside the packed nodal blocks. We preserve this case-sensitive mapping rather than treating every MATLAB quantity as another equal nodal field.

Hybrid inversion also changes the state handoff. An example can exclude or freeze the parameter being inferred from the EnKF stage, update the observed/state blocks, and subsequently pass the resulting member to its inverse solve. The returned parameter, model geometry, and diagnostic quantities must refer to the same completed analysis state before publication. The exact treatment depends on the example's hooks and activation schedule; a generic filtering flag does not supply an ISSM inverse problem automatically.

Table 1: Icepack and ISSM coupling boundaries across execution modes. These descriptions identify the implemented routes, while the tested configuration remains application-specific.

Mode	Icepack boundary	ISSM boundary
0	Generic single-driver field/array hooks and serial analysis.	One ICESEE lifecycle with an external model/server; internal solve resources remain separate.
1	Generic member forecasts; ordinary root ensemble/transform and distributed row update.	Member-worker/server workflow with complete-member transfer and shared analysis; generic topology limits apply.
2	File-backed generic forecasts and row analysis; full-member Python buffers can remain despite a spatial solver.	Complete-member external forecasts with file-backed shared analysis; no native spatial state is implied.
3	Specialized spatial PIG fields, or a distinct one-spatial-rank synthetic-stream binding.	Registered `ISMIP_Choi` whole-member round lifecycle and inversion; internal solver parallelism is separate.

4.5 Bringing a new application into ICESEE

We begin an application integration by defining its scientific state before choosing an execution mode. The state specification gives variable order, units, dimensions, prognostic versus diagnostic roles, uncertain parameters, observation equivalents, and the geometry/constraints required after analysis. A deterministic model-only experiment then provides a reference trajectory. This establishes what one forecast interval and one observation mean before uncertainty, collective communication, or restart is introduced.

The generic application template contains an example entrypoint, a configuration scaffold, and a model hook module in the example/package layout resolved by `SupportedModels`. We adapt its initialization and forecast calls to the selected model and pass member identity explicitly. Required input paths, time-grid construction, reference/synthetic observations, and outputs are declared alongside the example. A hook template is a starting point for that binding, not an interchangeable physics implementation: the existing adapters have some different signatures and return conventions, which are checked against their active callers.

A native extension additionally supplies owned field handles or an explicitly whole-member state, the member lifecycle, observation routing, and consistency operations. It registers an example-scoped Mode-3 runner or a supported shared native binding. Spatial ownership is validated for gaps, overlaps, and stable variable definitions. A reactivation callback and working-store choice are required for a streaming policy; a restore-compatible field/layout/policy definition is required for a genuine restart binding. Declared capabilities make unsupported combinations visible before expensive solves.

We validate the template progressively: distinguishable member initialization and block mapping, a known forecast, observation values/IDs, a small analysis against a numerical reference, actual multi-rank participation, and checkpoint continuation across an analysis event. A spatial adapter also exercises more than one rank per model and verifies that ghosts are not counted as owned state. A whole-member external adapter instead tests member/server identity across rounds. Only after these stages do we run a reduced target-physics experiment and evaluate its scientific diagnostics.

The companion guides `docs/handbook/coupling-icepack.md`, `coupling-issm.md`, and `application-onboarding.md` provide the command-level and template reading route. They keep model equations, backend installation, generic hooks, native registration, and experiment acceptance as related but distinct parts of onboarding. This organization allows students to develop an application against a small reproducible case while retaining a clear path to the larger parallel modes described next.

5 Hybrid inversion and parameter inference

5.1 The parameter and the inverse problem

We distinguish an ensemble parameter update from a deterministic model inverse. In joint filtering, parameter increments follow sampled relationships with the observed state. In the hybrid friction workflow, we instead update selected state variables through assimilation, then solve a separate ISSM control problem for each member. Control methods provide a framework for relating unknown model parameters to an observation-misfit objective (MacAyeal, 1993). The inferred basal-drag pattern also depends on the chosen ice-flow approximation and its boundary conditions (Morlighem et al., 2010). These dependencies remain part of the inverse experiment even when the model call is hidden behind a shared ICESEE hook.

The inspected `ISMIP_Choi` application orders six nodal blocks as

$$z_i = (H_i, s_i, u_i, v_i, b_i, C_i), \tag{14}$$

where C is the absolute ISSM `FrictionCoefficient`. It is not basal stress or a universally dimensionless drag multiplier. The selected friction law, exponents, unit conversion, and model approximation define how this coefficient enters traction. The prior-generation file stores a perturbation to which MATLAB initialization adds `mean_friction`; the inverse-stage member output instead stores the absolute coefficient, which is read directly. Applying that mean a second time would change every initial guess.

The six blocks have equal nodal sizes in this example. Python's zero-based `vel_idx=2` and `friction_idx=5` identify the velocity and coefficient positions. The separate basal-friction-variation example uses its own block names and layout; it is not obtained by relabeling the Choi coefficient. Changing a layout therefore requires checking observation selection, packing, physical finalization, and return mapping as well as the YAML list.

5.2 Objective, regularization, and solver settings

For one member at an active analysis time, the configured inverse problem is

$$\min_{C_{\min}\le C\le C_{\max}} J(C) = w_{101}J_{101}(C) + w_{103}J_{103}(C) + w_{501}J_{501}(C). \tag{15}$$

Here the identifiers refer to ISSM's finite-element response functionals: 101 is absolute surface-velocity misfit, 103 is logarithmic surface-speed misfit, and 501 is the drag-coefficient gradient penalty. The final term regularizes spatial variation of the control rather than its absolute magnitude. We retain the identifiers because quadrature, masks, normalization, small-speed treatment, and the exact gradient calculation belong to the installed ISSM implementation. Equation 15 states the active ICESEE application choices without replacing those responses by unverified continuum normalizations.

The application assigns `abs_vel_weight` to 101 and the historically named `rel_vel_weight` to 103. The latter does not select ISSM's relative-velocity response 102. Neither weight is automatically computed from the EnKF observation covariance. Thus an experiment combines an ensemble observation-error model with a separately weighted inverse objective; their common use of velocity observations does not make their weights statistically interchangeable.

Table 2: Active Choi inverse settings inspected in the MATLAB branch. These are application settings; convergence and installed-class defaults require the corresponding solver record.

Setting	Inspected value or operation
Control object	`m1qn3inversion()`, active `FrictionCoefficient` control
Responses and weights	`[101,103,501]`, weights $1, 1, 10^{-13}$ in the base YAML
Nodal coefficient bounds	$C_{\min} = 1900$, $C_{\max} = 4200$ in the selected parameterization
Evaluation limits	`maxsteps=40`, `maxiter=40`
Control/gradient criteria	`dxmin=0.1`, `gttol=1e-4`
Stress-balance criteria	`restol=0.01`, `reltol=0.1`, `abstol=NaN`
Model operation	`solve(md,'Stressbalance','runtimename',false)`
Internal process resources	ISSM `generic(...,'np',nprocs)`, separate from ICESEE worker ranks

The optimization limits do not report which criterion terminated an actual solve. A scientific record includes the objective components, iteration/gradient information, termination reason, bounds, and resolved inversion object. The application does not explicitly assign `incomplete_adjoint`, `control_scaling_factors`, or `dfmin_frac` after constructing the object. The official class inspected during this audit initializes these to one, but a local build has to be identified before those defaults are attributed to a run. The exact/incomplete-adjoint distinction is material in ice-sheet friction inversion (Morlighem et al., 2013); a generic reference to an adjoint does not establish that the current wrapper uses a complete one.

5.3 Analysis timing and the generic state handoff

The inverse is scheduled at observation events rather than every forecast. Generic Modes 1 and 2 retain the requested inversion switch and evaluate the effective activity as

$$I_\ell = I_{\text{requested}} \ \wedge \ \left(t_\ell^{\text{obs}} + 10^{-12} \geq t_{\text{start}}\right). \tag{16}$$

The helper uses the requested observation time when available and separately reports its mapped model-state time. This avoids deferring an intended year-two inverse merely because the nearest grid point lies slightly earlier. The observation slot ℓ, forecast cycle k, and physical time remain different quantities. There is no separate inverse-frequency control in these paths: after activation, inversion follows every due analysis while enabled. A forecast-only interval does not invoke it.

For the active generic hybrid cycle, the friction field is reserved for the inverse rather than updated first by the EnKF. Writing $\mathcal{F}$ for the configured non-friction finalization, the handoff is

$$z_i^\star = \left(\mathcal{F}[\mathcal{A}(H, s, u, v, b)]_i, C_i^f\right), \tag{17}$$

$$z_i^{\text{next}} = \left(z_{i,\neg C}^\star, \mathcal{I}_{\text{ISSM}}(z_i^\star, y_{u,v})\right). \tag{18}$$

This describes Python's exchange and overwrite policy, not a claim that every analyzed geometric field is assigned to the ISSM model before its solve. That separate limitation is discussed below.

Mode 1 saves the full forecast, removes the friction block from the analysis problem, updates/finalizes the other five blocks, and merges them back with forecast friction. It broadcasts the resulting complete ensemble for inverse preparation. Rank r processes members $r, r+P, \ldots$, while each Python callback receives a singleton communicator; the ISSM solve uses its independently configured internal resources. The generic writer accepts the returned coefficient and replaces only those rows. Returned geometry and velocities do not overwrite the analyzed values in that writer. Consequently this path can retain full-ensemble memory during inversion even when forecasts are scheduled member by member.

Mode 2 retains a fixed full-state file shape, excludes coefficient observation rows, temporarily freezes the coefficient analysis rows, and finalizes the reduced non-friction state. Its compatibility finalizer merges that result with saved forecast friction so a provisional EnKF coefficient increment cannot become the inverse initial guess. Workers read full members from an unpublished analysis shard, call the inverse, and validate the returned mapping, coefficient key, and length. Only after all workers report success are coefficients written and the new canonical state published atomically. The caller's original freeze policy is restored after the cycle.

This yields a precise publication guarantee: a failed inverse leaves the prior canonical forecast/restart shard unchanged in Mode 2. It does not roll back MATLAB files, subprocess state, solver scratch, or an external side effect already produced by a member. Mode 1's root writer and Mode 2's temporary publication also remain different transaction designs. Their numerical handoff can be compared without claiming identical failure recovery for every external object.

5.4 What MATLAB uses and returns

The Python hook writes the configured member blocks, selects the inverse command, and passes the observation slot for MATLAB's one-based observation-column lookup. The inverse branch loads its reference model and reads target velocities from the synthetic observation file. Analyzed velocities provide an initialization, not a replacement for those targets. The observation reader supports dense values or compact values with canonical zero-based IDs and optional activity masks. Sparse velocity coverage must still satisfy the installed ISSM consistency and objective-mask requirements.

Before solving, MATLAB reads analyzed u, v and absolute C, computes pressure from analyzed thickness, and forms an ocean level set from analyzed thickness and bed. It zeros

the initial coefficient on diagnosed floating nodes, restores the selected friction exponents, sets forcing/boundary options, and calls BAMG before configuring the control problem. Those steps make reference model, remeshing, nodal ordering, masks, and output sizes part of the coupled inverse experiment.

The inspected sequence has an important geometric qualification. Assignments of analyzed thickness, surface, bed, and base to `md.geometry` before the inverse solve are commented out. The stress-balance model therefore uses the loaded/reference and remeshed geometry, while its pressure and grounding-mask calculations use analyzed fields. Analyzed thickness/surface/bed are copied into geometry after solving. We consequently cannot describe the current branch as an inverse consistently posed on all analyzed geometry. Changing that sequence would require a scientific and numerical comparison, rather than a documentation correction.

After solving, the branch copies `StressbalanceSolution.FrictionCoefficient` into the coefficient, but leaves `md.initialization.vx/vy` assigned to themselves. It does not explicitly restore the optimized stress-balance velocities to those returned fields. Python subsequently reads the outputs in Fortran order. Generic Modes 1 and 2 use only the returned friction; they do not replace their analyzed velocity with a field presumed to be the optimized solution. Remeshing and a mapping of the expected original packed shape also require target-model validation. The inspected non-joint branch has additional variable-initialization limits, so the documented handoff refers to the selected joint-estimation Choi configuration.

5.5 Native inversion, failure propagation, and evidence

The Choi Mode-3 binding uses a whole-member adapter with one spatial rank. It calls the global native analysis without the generic row-control callback and supplies no global physical-analysis finalizer. Thus the coefficient can enter the full ensemble transform before inversion; the binding does not automatically reproduce the generic five-block reduction, friction freezing, localization, or SMB/bed plugin processing. Its inverse mapping also restores every returned named field, unlike the generic writers' coefficient-only overwrite. These differences are part of its application contract and prevent a blanket hybrid-parity claim from the common mode number.

The native framework snapshots registered fields immediately before inversion. If a callback reports failure, it restores those pre-inversion analyzed fields and raises collectively. This transaction does not revert the earlier analysis or external MATLAB files/context. A narrowly scoped later handover correction makes the Choi hook raise when the legacy inverse returns no mapping. Previously that missing result could be confused with the whole-member protocol's valid `None` return for deliberate in-place mutation. The correction changes exceptional failure propagation while preserving successful mapping updates; focused Python regression tests exercise it and the framework rollback. They do not execute a MATLAB inverse or validate optimizer convergence.

The native runner writes history shards and records requested inverse activity, but it has no complete coupled checkpoint-load/resume binding. Scheduled-operation metadata and a written coefficient file are therefore separate from demonstrated inverse convergence or restart. The source map and companion inversion guide provide exact hook, MATLAB branch, control, and test locations. A reproducible scientific experiment additionally retains the reference model, resolved objective, observation support, coefficient prior/bounds, solver versions, convergence diagnostics, and a held-out or known-truth evaluation.

6 Physical inference and supporting developments

6.1 Apparent mass balance from an analyzed history

The optional inference plugin contains post-analysis operations distinct from the friction optimizer. Its SMB physics route retains a short analyzed history for each member and estimates the thickness tendency from physical time. With centered times $\widetilde{t}_j = t_j - \bar{t}$, it forms

$$\widehat{\dot{H}} = \frac{\sum_j \widetilde{t}_j H_j}{\sum_j \widetilde{t}_j^2}, \tag{19}$$

$$r = \widehat{\dot{H}} + \nabla \cdot \left(\overline{Hu}, \overline{Hv} \right). \tag{20}$$

The temporal fit requires at least three recorded times, with a default window of five. Flux derivatives use weighted quadratic moving-least-squares reconstructions on neighboring physical coordinates. Coordinates are scaled to metres and depth-averaged velocities are interpreted in metres per model-time unit, so the inferred balance uses the same time unit as the retained history.

For a positive basal-loss convention, Eq. 13 gives $r = a_s - a_b$. The helper does not add an independently inferred basal term. We therefore interpret this residual as apparent/net balance unless basal loss is zero or independently accounted for. Assimilation jumps can also contribute to a slope fitted through analyzed states. A time window smooths that behavior but does not convert every analysis increment into a physical mass-balance signal.

The inferred field solves a regularized problem of the form

$$\widehat{q} = \arg\min_q \left\{ \tfrac{1}{2}\|q - r\|^2 + \tfrac{\lambda_s}{2}\|Lq\|^2 + \tfrac{\lambda_t}{2}\|q - q_{\text{prev}}\|^2 \right\}, \tag{21}$$

with normal equations

$$[(1 + \lambda_t)I + \lambda_s L^T L]\widehat{q} = r + \lambda_t q_{\text{prev}}. \tag{22}$$

The symmetric dimensionless graph Laplacian L uses symmetrized nearest-neighbor Gaussian weights and median-positive-degree scaling; it annihilates constants. Defaults are $\lambda_s = 25$, $\lambda_t = 4$, twelve graph neighbors, and 24 requested divergence neighbors. These regularization weights are not observation-error covariances. The helper blends inferred and analyzed SMB with default factor 0.35, subject to configured activation/ramp and a spatial basis.

If SMB is directly declared in `observed_params`, a separate branch treats it as already conditioned by filtering: it projects the analyzed posterior and clips it, without fitting Eq. 20. In the physics branch, inferred values are clipped before blending and basis projection, without a final clip after those operations. Consequently a configured SMB bound is not a guaranteed final bound in every branch. This ordering and the selected basis are part of the method, not cosmetic output processing.

6.2 Bed increments, survey support, and anchors

The bed helper regularizes an increment proposed by the filter or another stage, rather than solving the velocity-based friction inverse or smoothing the absolute bed repeatedly. For proposed bed b_a, previous/reference bed b_p, and optional mask weights M clipped to $[0, 1]$, it forms

$$d = b_a - b_p, \qquad (I + \lambda_b L^T L)d_s = Md, \qquad b_{\text{new}} = b_p + \beta_b d_s. \tag{23}$$

The implementation can project the smoothed increment onto a selected basis and cap it before blending. Defaults are $\lambda_b = 40$, twelve graph neighbors, and $\beta_b = 0.15$, with optional time-dependent blending. Final operations enforce configured bed bounds and $b \le s - \delta$, with default separation $\delta = 1$ metre when a surface block exists. This safeguard is not a complete flotation

or stress-balance solve. Increment regularization preserves a rough prior unless the proposed increments actually change it.

Survey cadence uses observation slots and `bed_snap_cols`. The legacy policy restores forecast bed outside survey columns; explicit snapshot-only activity requires a nonempty survey list; continuous activity retains cross-covariance updates at every analysis. Disabling the new plugin retains the legacy gate. Restricted grounded/observed support is applied by the inspected global gate at survey columns, rather than automatically masking every nonsurvey continuous update. The grounded test uses each forecast member, while observed support follows the stored survey mask.

Optional deterministic bed anchors apply $b_i \leftarrow b_i + \eta(y_b - b_i)$ on active finite observations, with separate pre/post factors in $[0, 1]$. At that operation the innovation contracts by $1 - \eta$, while unobserved locations are unchanged. Anchoring adds a deterministic correction to the stochastic posterior. Its factor, support, and position in the processing sequence are therefore recorded with the experiment.

6.3 Control order, persistence, and diagnostic scope

The shared generic row policy relaxes and optionally caps increments, inflates anomalies about the controlled mean, restores frozen rows, and applies the local bed schedule. Global finalization then handles bed support/anchors and references, bed regularization, geometry where implemented, and SMB correction after geometry. A freeze at one stage is not a universal prohibition on later independently enabled callbacks. Equal-sized blocks are required by these shared routines, and a native segmented binding needs an explicitly compatible policy.

Mode 2 preserves private bed/SMB history state across copied runtime mappings. That object-lifetime persistence is different from durable restart of temporal references. Ensemble-coupled finalization can also require a complete ensemble under a separate budget even when the ordinary analysis is row-bounded. The specialized PIG native global policy does not automatically call these gathered generic helpers. We evaluate an inference setting through its actual callback path and state lifetime.

Table 3: Supporting developments and their present evidence boundary. The mechanisms are source-inspected capabilities; the table does not assign a first-introduction date or imply physical validation of every option.

Mechanism	Purpose	Evidence/remaining boundary
SMB/bed inference	Regularized apparent balance or bed increments	Bed/support contracts tested; a dedicated manufactured-SMB suite was not found.
Analysis controls	Shared relaxation, caps, inflation, freezes and gates	Representative row/hybrid tests; native policy must bind them.
YAML/CLI resolution	Child inheritance, typed overrides and effective-setting checks	Loader/override tests; application time/unit setup remains explicit.
Structured diagnostics	Wall/rank time, measured/unmeasured phases and backend I/O metrics	Reporter tests; overlapping counters are not additive wall-time fractions.
Graph/JIT coordination	Cache deterministic geometry; coordinate compilation participants	Cache/random-state and communicator tests; backend-version structure still matters.
Time/output identity	Clock markers, owned coverage, committed shards and closed-history export	Targeted recovery tests; each application still needs its restore binding.

Recursive YAML inheritance and checked CLI overrides preserve the distinction between requested and effective scientific settings. Structured performance reports distinguish an unmeasured phase from no events or a measured zero. A 64-MiB per-process stochastic graph cache stores deterministic coordinate/neighbor work, not sampled random fields, and includes coordinate contents in its key. PyOP2 cache coordination makes hit/miss decisions consistent on its compilation communicator before collective compilation. These developments support reproducibility and diagnosis; they do not change a regularization prior into independent observations or make unperformed physical validation implicit.

7 State ownership and process topology

7.1 Replicated and distributed state are different contracts

A model can return a complete member on every rank in its model communicator, or disjoint owned pieces of that member. `src/utils/state_ownership.py` makes this difference explicit. For replicated state, the global member dimension equals one local dimension; concatenating ranks duplicates the state. For distributed state, the global dimension is the sum of disjoint owned dimensions. Array shape is not enough to determine which contract holds.

The small Lorenz example is replicated. Its directory and function names say `lorenz96`, but its right-hand side is the three-variable system

$$\dot{x} = \sigma(y - x), \qquad \dot{y} = x(\rho - z) - y, \qquad \dot{z} = xy - \beta z. \tag{24}$$

These are Lorenz-63 equations. We retain existing path names for executable compatibility and identify the actual equations in the scientific description. Its integration tests establish properties of this small implementation, not of a high-dimensional Lorenz-96 system.

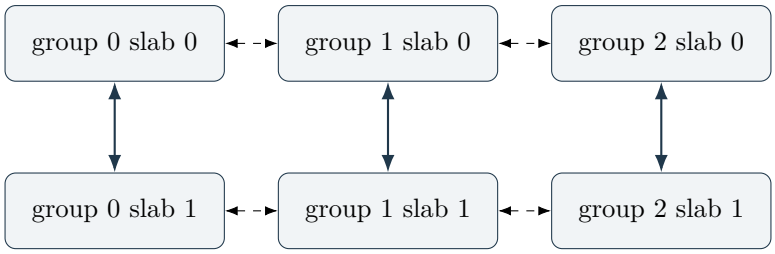
In finite-element models, a local vector can include owned entries and ghosts. Only owned entries belong in a global state partition. Ghost entries support stencil or finite-element communication and must not be counted a second time. A segmented state layout describes each named variable separately, preserving variable-major packing while concatenating only the locally owned pieces. The partition validator must detect gaps and overlaps for every variable.

7.2 Two levels of parallel work

Native distributed execution separates model groups from the ensemble communicator. A spatial communicator joins ranks that cooperate on one member's forecast. An ensemble communicator joins corresponding spatial positions across groups and assembles the same state rows for different members. More members than groups can be scheduled in rounds. Thus $P = N_{\mathrm{e}}P_s$ is not a general requirement; the topology must permit a group to handle several members.

Node locality is an operational property, not a consequence of a mode number. PIG supports node-local native model groups, and a retained topology check confirmed locality for the five-year experiment. MPI launch order, scheduler rank placement and the ranks-per-model value must all agree. The arithmetic group split and the scheduler placement are therefore checked together on a multi-node allocation.

Every collective operation has a participant set specified by its communicator (MPI Forum, 2025). A collective HDF5 open on a model communicator requires all ranks in that communicator to participate. A model-group-root-only write cannot safely trigger a world- or shared-communicator collective. Earlier repairs separated collective window preparation from root-only writes. This rule generalizes to barriers, broadcasts, close operations and recovery: after one rank fails, arbitrary further collectives can hang.



Solid: spatial communication within a model group
Dashed: matching owned rows across model groups

Figure 1: Conceptual native process grid. Group identity is distinct from member identity; groups may advance multiple stable member IDs. This sketch is not an assertion that every adapter uses the same placement.

8 Execution modes and parallel work distribution

We distinguish the ownership of forecast members, observation factors, state rows, and output. Figure 2 summarizes the four principal execution paths. The descriptions below follow the active code, including alternate branches where their memory or capability boundary differs from the ordinary path. They identify where the numerical operations occur, rather than ranking the modes by their integer value.

Mode 0: serial lifecycle
One process initializes and forecasts members, constructs serial analysis inputs, updates the ensemble, and writes history.

Mode 1: member forecasts and row updates
Model groups forecast → leaders gather members at root → root forms T → broadcast T, scatter state rows → row updates and shared output.

Mode 2: file-backed ensemble
Groups forecast into shards → observation-row chunks form reduced ensemble products → each rank solves T → streamed state updates, bounded assembly, and publication.

Mode 3: application-specific native lifecycle
Native spatial groups retain fields → matching rows communicate across ensemble groups → reduced products and transform → owned-field update and sharded checkpoint.

Figure 2: Principal execution paths. Mode 1's partitioned alternative and Mode 3's whole-member bindings have additional qualifications described in the text. The mode selects orchestration and representation; the filter and scientific policies remain separate choices.

8.1 Quantities that must remain distinct

Let `n` denote the packed state dimension, `Ne` the number of ensemble members, `m` the number of active observations, `P` the number of ICESEE MPI world ranks, and `S` the ranks cooperating on one model forecast. A member ID is a scientific identity in `0,...,Ne-1`; an MPI rank identifies a

process. One process can advance several members over successive rounds, and several processes can cooperate on one member. We therefore do not equate the ensemble size with the MPI size.

We arrange an ensemble as an `nxNe` matrix with members in columns. **Member distribution** assigns different columns to workers. **State-row distribution** assigns different rows to analysis workers. **Spatial model decomposition** assigns owned mesh degrees of freedom to collaborating model ranks, including the solver's halo communication. A row slab used in a matrix multiplication is not automatically a valid spatial partition for a PDE solver.

Table 4: Execution, scheduling and storage options are distinct choices.

Quantity or option	Meaning in the active code
`execution_mode`	Chooses the serial, partial-parallel, file-backed, or application-registered native runner.
`filter_type`	Statistical method name. Its presence in the parser does not establish support in every runner.
`default_run`, `sequential_run`, `even_distribution`	Historical scheduling selections within the configuration/manager layer. Their end-to-end wiring must be checked in the chosen driver.
`sequential_ensemble_initialization`	Initialization policy only; it does not mean that all later forecasts use `sequential_run`.
`ranks_per_model`	Generic resource-plan request for model-group size. Omission preserves the planner's legacy/default policy.
`model_nprocs`	Application solve setting. Native PIG uses it for spatial MPI groups; ISSM can use it for an internal external-model solve.
`state_distribution`	Whether the generic model-array boundary is replicated or consists of disjoint distributed pieces.
`analysis_backend`	A lower-level option in the serial EnKF class; it is not the canonical top-level runner selector.
`batch_size`	Mode-2 open-file window in state columns, not an ensemble mini-batch or a model timestep.

In a replicated model, several model ranks may hold the same complete state; its global dimension is one local dimension. In a distributed model, owned pieces combine to cover the member once. Ghost values do not add new state rows. The Lorenz example declares replicated ownership, whereas native Firedrake layouts describe owned entries and synchronization explicitly. This distinction must be established before assigning a larger `S`.

8.2 How the generic resource planner assigns work

`plan_resources` computes a plan without launching MPI. If `ranks_per_model` is omitted, it chooses `S=1` when `Ne>=P`; when `P>Ne`, it uses `S=max(1,P//Ne)`. Explicit `auto` uses the latter floor-division rule, and a positive integer requests exactly that group size. The planner then forms at most `min(P//S,Ne)` groups, optionally limited further by a model-group cap such as the ISSM server count. Ranks after the last complete group are spare. Groups are block-contiguous in world-rank order.

For `G` groups, round `r` assigns member `g+r*G` to group `g`, provided that ID is smaller than `Ne`. Thus the number of rounds is `ceil(Ne/G)`, and the final round can contain idle groups. This is a statement about the plan. Each runtime path must also implement those rounds, spare-rank guards, and collective participant sets correctly. The Mode-2 forecast routine has an explicit unified round loop; some older Mode-1 branches still distinguish `Ne>=P` from `Ne<P` and should not be assumed to implement every possible plan.

8.2.1 More members than ranks with the default single-rank grouping

For `P=8`, `Ne=12`, and omitted `ranks_per_model`, the generic plan has eight one-rank groups and two rounds:

Table 5: Default assignment for eight one-rank groups and twelve members. Columns 0–7 identify group/rank IDs; entries are member IDs.

Round	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	8	9	10	11	idle	idle	idle	idle

This is the ordinary default forecast layout for the generic parallel paths in this regime. We have not obtained two spatial ranks per member merely by having eight ranks and twelve members. The active members are independent solves, each on one rank; after both rounds their results participate in the same twelve-member analysis.

For `P=Ne=8`, the same default gives eight one-rank groups and one round. For `P=10`, `Ne=4`, omitted grouping gives four two-rank groups and two spare ranks. The latter case requires both replicated/distributed ownership handling and safe treatment of ranks that belong to no model communicator. Mode-2 tests exercise such cases. A planner's valid mapping is not, by itself, evidence that another driver's initialization and forecast branches safely handle its spare ranks.

8.2.2 An explicit spatial grouping

With `P=8`, `Ne=12`, and an adapter-supported explicit `S=2`, there are four groups and three rounds:

Table 6: Four two-rank spatial groups process twelve members in three rounds.

Group	World ranks	Members processed in order
0	0, 1	0, 4, 8
1	2, 3	1, 5, 9
2	4, 5	2, 6, 10
3	6, 7	3, 7, 11

The native PIG topology can express this layout through its own `model_nprocs=2` setting. We use this example below for Mode 3. The table is not a universal instruction to add `--ranks_per_model=2` to an arbitrary model: the manager validates registered capabilities, stochastic decomposition requirements, and, for selected models, node-local placement. ISSM's internal MATLAB/PETSc process count is a different boundary and must not be substituted for a generic spatial communicator size.

8.3 Mode 0 serial orchestration

Mode 0 calls `icesee_model_data_assimilation_serial`. We use this path to make the complete assimilation cycle accessible on a small problem and to investigate serial filtering alternatives. The runner assigns its own ICESEE rank/size as zero/one. Launch it as one Python process. Launching several copies with `mpirun` does not convert it into Mode 1 and can create multiple apparent root writers.

8.3.1 Initialization, forecast, and analysis

The runner resolves the model hooks, generates or reads truth/reference trajectories and synthetic observations, and calls the serial ensemble initializer. The resulting initial member values are saved at state column zero. At cycle `k`, it reads the complete current ensemble `ensemble[:,:,k]` from `icesee_ensemble_data.h5`. The EnKF class's serial forecast backend iterates over members, calls the model's forecast hook, and applies its configured member-wise process-noise policy. It then writes forecast members and their mean to column `k+1`.

When an observation is due, the same process reads observations, constructs the active observation covariance/operator inputs, and performs the selected analysis. The serial driver has explicit branches for `EnKF`, `DEnKF`, `EnRSKF`, and `EnTKF`. For EnKF/DEnKF it constructs a dense state covariance from ensemble anomalies before calling analysis; this stage can require $O(n^2)$ storage. A transform-based implementation elsewhere in ICESEE does not make this serial driver universally covariance-free. Serial availability also does not establish that each alternative method has been validated for every model and observation operator.

After analysis, an optional model callback restores physical consistency. Members and means are written to the completed state column, and the next cycle reads that analyzed state. Thus one process controls model calls, observation preparation, analysis, and output. The underlying model can still use its own external solver processes, but those are not ICESEE ensemble workers.

8.3.2 Memory, communication, and use

There is no ICESEE member distribution or state-row scatter in the default serial backend. MPI may still be imported for timing and common utilities, so serial execution is not a promise that mpi4py is absent. The current ensemble and analysis objects must fit one process. HDF5 history avoids retaining every state column in one Python array during the loop, but the initializers and selected analysis can allocate additional full-sized arrays. Do not size this mode using only the final file's dimensions.

Mode 0 is the appropriate first step for the small student exercise, an operator/observation mapping investigation, or a serial-method comparison. It does not establish multi-rank collective safety. Its lifecycle also does not provide the specialized PIG native checkpoint manager; do not infer a supported arbitrary restart from the existence of a common `restart_enabled` key.

8.4 Mode 1 partial parallel orchestration

Mode 1 calls `icesee_model_data_assimilation_partial_parallel`. In its ordinary `default_run` path, forecasts are distributed over model groups, while the global analysis transform is constructed on world rank zero. State-row multiplication is then shared over world ranks. We call this *partial parallel* because not every stage has the same distribution; it does not mean that only the forecast has parallel work.

8.4.1 Initialization and forecasting

The manager creates model communicators from the resource plan. Reference generation and observation setup have designated writers and synchronization, and the MPI initializer constructs members using stable member IDs and model hooks. The `sequential_ensemble_initialization` setting can serialize setup even when subsequent forecasts are parallel. This can be useful for adapters whose expensive setup or file access cannot be safely replicated; it should be recorded with initialization timings.

For `P=8`, `Ne=12`, default single-rank grouping, the forecast routine advances members 0–7 in the first round and 8–11 in the second. Each active group reads its member's current state, calls `advance_ensemble_member`, and obtains its updated values. Group leaders send member IDs and completed vectors through a leader communicator to world rank zero. The receiving root places each vector into the correct column of the full forecast ensemble; arrival/rank order must not replace member identity.

The ordinary forecast function allocates a complete `nxNe` result array, and world rank zero needs the assembled ensemble. Some non-root paths also allocate full-shaped work arrays. We therefore do not claim per-rank `n*Ne/P` memory for Mode 1. If a model uses several ranks per solve, its actual owned/distributed return convention must still be honored; full-vector transfer at the generic boundary can remain even when the solver itself is parallel.

8.4.2 Observation work, transform, and state update

At an analysis event, the ordinary path loads observation values and masks, and world rank zero calls `EnKF_X5`. It evaluates forecast equivalents, forms the configured observation-error factor, and constructs the small global transform `X5`; grouped local transforms can also be prepared there. The selected factor matters statistically: `legacy_prior_anomalies` reuses forecast anomalies, while `stochastic_R` and `generated_R` use explicit perturbation factors. Those choices are described in numerics.

`analysis_enkf_update` broadcasts `X5` and the local-patch registry. It divides state rows with balanced counts, then `Scatterv` sends each rank an `n_localxNe` forecast slab. Every rank computes `forecast_slab@X5`, replaces selected rows with grouped local transforms where requested, and applies the shared row-local analysis controls. The writer then handles output and any model/global finalization. This distribution changes at the forecast/analysis boundary: ranks that owned particular members during forecast now work on rows spanning **all** members.

The root's factor construction and SVD retain observation-by-member arrays, approximately `O(m*Ne)` each, in addition to the complete state ensemble. Some setup/driver paths load dense observation histories on other ranks as well. Calling the result a small transform therefore describes the final `NexNe` matrix, not the entire memory used to construct it. This is the important difference from Mode 2's row-streamed product construction.

This scheme follows the useful separation between ensemble-space algebra and state-space application: a small member-space transform can be reused across many state rows. Ensemble representations and numerical implementation choices are discussed by Evensen (2003). That literature provides context, while ICESEE's exact finite-sample factor and truncation remain defined by its code.

8.4.3 Partitioned I/O and limits

`partitioned_io_flag=True` selects a separate Mode-1 path. Forecast members are written directly to shared HDF5 rather than gathering a full ensemble for the ordinary root path. The partitioned analysis routines read distributed rows and reduce analysis products collectively. This is an explicit alternative within Mode 1, not another name for Mode 2; its supported options and finalization boundaries must be checked separately. For example, the partitioned EnKF branch rejects a restricted `bed_update_domain`, and the driver explicitly refuses partitioned DEnKF.

The active production explanation here concerns EnKF with `default_run`. Although Mode 1 recognizes other filter names and contains DEnKF-related routines, branch presence is not evidence of a working, parity-tested configuration. The current DEnKF call interfaces require independent validation before use. Likewise, the general planner can represent spare ranks or additional rounds, but older Mode-1 initializer/forecast branches still use `Ne>=P` tests and direct subcommunicator queries. Use a demonstrated topology, rather than assuming every mathematically valid plan executes safely in this path.

8.5 Mode 2 file-backed and bounded-row analysis

Mode 2 calls `icesee_model_data_assimilation_full_parallel` and uses `EnKF_fully_parallel_IO`. Here we retain ensemble state in timestep files and stream observation/state rows during analysis. This removes the ordinary Mode-1 requirement that world rank zero assemble the complete forecast ensemble for global analysis. It does not remove the state needed by an individual generic model forecast.

8.5.1 Initialization and model forecasts

Every world rank constructs the I/O layer and participates in its required shared setup. The full-parallel initializer writes the initial state at shard zero. Depending on the initialization

policy, members can be initialized in groups or prepared by root and written through a collective root-to-row distribution. A rank-zero reuse decision is broadcast so ranks do not disagree about whether to initialize or enter later collectives.

The active file-backed cycle uses `default_run`. It follows the centralized plan in a unified round loop for both `Ne>=P` and `Ne<P`. Active groups read one current member from the shared state shard, advance it, and write its completed state at shard `k+1`. A distributed generic model can update only part of that vector on each model rank; the routine gathers the freshly owned pieces to the group leader before its full-member write. Replicated model copies do not need that assembly. This is why Mode 2 can bound ensemble-analysis memory while still using a full-member array on a group leader.

The generic `read_forecast` returns a complete member vector to its caller. Consequently model ranks can retain an `O(n)` NumPy buffer even when the PDE solver uses a spatial communicator. We should not infer `O(n/S)` Python-state storage from solver parallelism alone; the native field boundary in Mode 3 addresses that separate representation issue.

Spare world ranks do not enter model callbacks or call methods on `COMM_NULL`. They do, however, join the world-level file-window priming operation and other shared I/O-layer collectives. Groups with no member in the last round also remain consistent with those participant sets. The primed window covers the state being read and the state being written; subgroup-root writes must find it already available rather than initiating an unmatched collective open.

The I/O class's `self.nt` is the number of **saved state columns**, set to the driver's forecast count plus one. The driver loops over forecast cycles `0,...,nt-1`; the final advanced state is shard `nt`. When following index arithmetic inside `compute_analysis_update`, first account for this constructor normalization. Otherwise a correct `k+1` mapping can be mistaken for a terminal-time exception.

8.5.2 Observation rows and the small transform

The indexed observation operator records selected state rows rather than allocating a dense `mxn` matrix. Compact synthetic observations store active observed values, standard deviations, IDs, and masks. The analysis divides observation positions among world ranks and reads only each rank's active row chunks from the immutable forecast shard. For a global EnKF update, every observation contributes once to three `NexNe` products:

```
F = Yprime + Eta
C_local += Yprime.T @ F
G_local += F.T @ F
B_local += F.T @ D
```

Three `Allreduce(SUM)` operations form the global products. Each participating rank then computes the small spectral transform locally; this differs from the ordinary Mode-1 root computation followed by broadcast. We do not communicate a state covariance. The exact transform is `I+C@Gplus@Gplus@B`, with spectral truncation, as derived in the numerical chapter.

To preserve the canonical random sequence for explicit observation factors, rank zero writes an observation-sized temporary factor file in the configured canonical order. Other ranks read their row portions after synchronization. Thus global analysis is row-distributed, while factor preparation still includes root work and I/O. For `generated_R`, the field generator can also require a state-sized per-column workspace. Claims about bounded analysis must name these auxiliary stages rather than treating every temporary as `NexNe`.

8.5.3 State update and publication

After products are reduced, each rank reads chunks within its assigned state-row interval, multiplies those rows by `X5`, applies local patches and shared controls, and writes a private

serial analysis shard. Bounded assembly combines these rank shards; application finalization, geometry/inference, and optional member-wise inversion occur before atomic publication of the analyzed state. Means derived after finalization refer to the same published state used for restart.

Not all analysis branches have the same bound. Exact grouped local analysis in this I/O class gathers active observation factors to rank zero, checks `local_analysis_memory_budget_mb`, constructs the grouped transforms there, and broadcasts the patch registry. Some inference/finalization operations also require member slabs or ensemble-coupled root work under separate budgets. Reducing `analysis_row_chunk_size` does not reduce those independent allocations.

8.5.4 Retention, restart, and communication requirements

Full history costs approximately `n*Ne*(nt+1)*b` stored bytes before means, truth, checkpoints, temporary files, and overhead. Rolling history retains the restartable state and active window, so retained member history does not grow indefinitely with run length. It cannot reconstruct pruned historical spread. `auto` can choose retention from projected capacity; set and verify an explicit policy when the experiment requires complete members at every time.

The driver writes its completion checkpoint before pruning a rolling shard. Full-history finalization can publish a VDS over timestep sources. Preserve those sources and their layout when archiving. The minimum two-shard window is an I/O safety/working-set decision, not a reduction in ensemble size.

Parallel HDF5 has specific collective requirements on the file-opening communicator. Some metadata operations require coordinated calls, whereas raw dataset reads/writes can use independent or collective transfers. It is therefore inaccurate to say that every HDF5 read is collective, or that a root-only dataset write can safely perform any metadata operation it happens to need. See the HDF Group collective-call requirements (The HDF Group, 2026) and h5py parallel-HDF5 guide (h5py developers, 2026a). ICESEE's window preparation and immutable analysis-reader phases enforce this distinction in the selected path.

The maintained Mode-2 lifecycle described above is `default_run` with EnKF. The parser/manager retain historical sequential/even selections and the driver recognizes additional filter strings, but its active forecast/analysis block is not equivalently implemented for all those choices. Use the actual full-cycle tests, not a recognized CLI option, as the support criterion.

8.6 Mode 3 application-specific native execution

Mode 3 first calls `icesee_da_distributed.py`, which looks up an application-scoped registration. It then delegates to that application's runner. We use this mode to retain native model fields and, where the adapter genuinely exposes them, spatial ownership throughout the forecast/analysis interface. A registration identifies a lifecycle; it does not make all registered applications spatially distributed or scientifically equivalent.

8.6.1 Native topology and persistent fields

`create_distributed_topology` builds a rectangular `P=G*S` grid. `spatial_comm` contains all spatial ranks of one ensemble group. `ensemble_comm` contains corresponding spatial positions across groups. The native topology has no spare-rank remainder: the requested dimensions must factor the world size, and the native pool rejects an ensemble slot with no member. This differs from the generic planner's ability to leave spare ranks outside model groups.

For native PIG with `P=8`, `Ne=12`, `model_nprocs=2`, the four spatial groups and member assignments are those in the explicit-group table above. `ensemble_comm` at spatial position zero joins ranks 0,2,4,6; the communicator at spatial position one joins ranks 1,3,5,7. Each spatial group forecasts its three members. A `NativeDistributedMember` retains model fields and an

opaque context, while the field registry packs only owned entries when analysis/checkpoint code needs them. Ghost synchronization remains the model adapter's responsibility.

Initialization constructs or restores these native objects and validates exact state coverage. Serial reference observations can have a different mesh numbering from distributed model rows, so PIG maps physical coordinates to canonical observation positions before routing values or process-noise increments. Matching vector length alone would not establish that the same locations are being assimilated.

8.6.2 Forecast, observation reduction, and analysis

In `run_native_global_analysis_cycle`, the pool advances assigned native members using each spatial communicator. It evaluates locally owned observation equivalents, then assembles the selected rows across `ensemble_comm` in stable member-column order. Ranks sharing a spatial position must agree on their active observation IDs and batch order, while different spatial positions can own different observation batches.

Only ensemble slot zero accumulates the local observation products. Those products are reduced across its `spatial_comm`, avoiding counting each observation once per ensemble group. The small transform and any collective refusal reason are then broadcast across corresponding `ensemble_comm`s. State-row blocks spanning members are exchanged across ensemble groups, transformed, and written back to the owned native fields. An optional application callback can modify geometry or diagnostics at finalization. The current native PIG binding performs ghost synchronization without a state-modifying finalizer; its next forecast recomputes geometry and diagnostic velocity. Optional inversion likewise requires the appropriate adapter and analysis policy.

The diagram describes the native global-analysis building block, not every registered runner. There is also a grouped-local native cycle and a whole-member file-backed analysis policy. The choice belongs to the application's binding and scientific feature set. The local ensemble transform Kalman filter of Hunt et al. (2007) is a distinct local deterministic filtering method; its name should not be used as a synonym for this process grid or ICESEE's selected stochastic factor update.

8.6.3 Persistent members, streaming, and the remaining memory costs

By default, `NativeDistributedMemberPool` retains every member assigned to a slot. In the twelve-member/four-group example that means three native member contexts per group, not just the one currently being advanced. Native ownership removes a complete-member gather in the spatial PIG path, but the retained number of local contexts can still dominate memory.

`use_member_streaming=True` can select a streaming pool when the adapter implements reactivation. It keeps at most one active native member and stores the other assigned members' packed owned arrays. A memory store removes inactive solver/context overhead but still retains their packed state in RAM. `member_store_backend=hdf5_member_major` moves that backing to files; `use_store_streaming_analysis` additionally selects row-wise store analysis where the application wires it. These are separate controls, and an adapter without reactivation is not made streaming merely by a flag. Working member-store files are not committed restart checkpoints.

PIG's canonical process-noise decorator retains another limit: a group leader generates a global-sized stochastic increment, then scatters owned pieces. This is not a gathered physical member, but it is `O(n)` workspace with a 256 MiB guard. Its graph cache has a separate 64 MiB per-process limit. Spatial native execution should therefore be described by the bound of each stage, not by an unconditional assertion that no process ever allocates anything global-sized.

8.6.4 Application capabilities and output

Table 7: Native application boundaries at the inspected revision. Registration does not imply equal capabilities.

Registered application	Implemented boundary and principal restrictions
Lorenz `lorenz96` example	Tiny whole/replicated state; specialized native runner; legacy and stochastic-R factors; gathers small snapshots for legacy HDF5 output. Not evidence for distributed glacier fields.
Icepack `idealized_pig`	Native Firedrake owned fields; spatial groups, specialized checkpoint/replay and history export; legacy and stochastic-R factors. Current PIG runner uses its bound global-analysis policy; additional generic native/local infrastructure is not an automatic feature promise for this example.
Icepack `synthetic_ice_stream`	Whole-member adapter with one spatial rank; shared native lifecycle plus `FileBackedNativeAnalysis`, reusing the file-backed analysis operator for declared localization/inference features. Shared lifecycle rejects requested resume.
Flowline `flowline_1d`	Specialized small-model/whole-member lifecycle; sharded native history; legacy and stochastic-R factors; no model inversion mechanism in that binding.
ISSM `ISMIP_Choi`	Complete-member Python/MATLAB boundary; round scheduling and inversion policy; internal ISSM solve ranks are separate. Writes checkpoint shards but has no native restore path. Persistent local member arrays are not reduced to one by a streaming flag.

Registration is not evidence that every entry has the same tested `Ne`, process count, restart policy, filter support, or real-data skill. In particular, the shared `native_application` policy enforces one spatial rank, rejects resume, and rejects scientific feature flags unless the binding declares them. Specialized runners must be read and validated separately. Current native runner error-factor checks also mean that the presence of `generated_R` in generic Mode 2 should not be projected onto every native application.

Native checkpoint manifests bind member identity, layout, progress, and policy to rank-owned shards. For a runner with implemented restore, a completed cycle `k` resumes at cycle `k+1`, after analysis already applied to that saved state. The current ISSM Choi native runner writes checkpoint shards but has no load/restore/resume path; checkpoint creation alone is not restart support. Native PIG can export retained checkpoints into a standalone aggregate history in bounded rows. Other adapters can use a different output schema; an existing notebook expecting the generic HDF5 layout will not necessarily read their native shards directly.

8.7 Choosing and comparing modes

We choose the mode from the model boundary and the part of the workflow that limits the intended experiment. A fast analysis cannot compensate for a single forecast that does not fit its model ranks, and a spatial solver does not by itself provide a bounded complete history.

Table 8: A starting execution path and the evidence needed for the selected case.

Situation	Starting choice	Evidence required before larger use
Learn a new adapter/operator on a small state	Mode 0	Initial state, forecast and analysis shapes, observation timing, physical consistency.

Situation	Starting choice	Evidence required before larger use
Compare serial filtering variants	Mode 0, selected serial method	Method-specific numerical tests and relevant model experiment; do not infer parallel availability.
Parallel forecasts with a manageable current ensemble	Mode 1 EnKF/default path	Target-model/topology completion and member-wise control comparison; inspect root memory.
Full ensemble history is too large for the ordinary memory path	Mode 2 EnKF/default path	Full/rolling policy, capacity, coherent MPI/HDF5 stack, same-input Mode-1 parity, and restart/output verification.
A physical PIG member needs native spatial decomposition	Registered PIG Mode 3	Supported spatial layout, node-local groups, owned-field/observation mapping, spread, restart and physical acceptance.
Many round-assigned native contexts exceed RAM	A supported native streaming/store policy	Activation/store/analysis traffic measurements and numerical/restart equivalence; not just a reduced context count.
Another model or an unlisted feature combination	Established model-specific baseline	Inspect registration/capabilities and demonstrate the complete requested cycle before claiming support.

The companion handbook supplies the complete small YAML and paired Mode-1/Mode-2 harness. That harness fingerprints source and configuration, then retains full float64 histories for member-wise comparison. The PIG application supplies a separate input, environment, and batch procedure. These are application experiments sharing an orchestration library, rather than interchangeable launch commands.

Same-input Mode-1/Mode-2 parity requires the same realized prior, observations, member IDs, seeds, process-noise schedule, factor mode, controls, solver settings, and storage precision. Compare members and means, the observation/state clock, and the post-analysis physical fields. Mode 0 has a separate serial analysis path, and native initialization/noise can differ from generic initialization; equal filter labels do not make all four results a single controlled parity experiment.

Finite-ensemble covariance estimates remain sensitive to observation support and sampling. Communicator layout and row streaming address computational ownership, while localization, uncertainty, and posterior behavior remain scientific questions.

Finally, distinguish operational measurements from scaling experiments. A Mode-3 run with sixteen ranks and two ranks per model cannot establish speedup over a concurrent eight-rank Mode-1 run. Hold the scientific problem, initialization, solver stack, retention and I/O protocol fixed; define which process-grid dimension changes; record repeated wall times, per-rank memory, and storage traffic. Report simulation, output publication, and verification separately. The available production records establish their recorded configurations; a new application or scientific setting requires its own comparison and physical evaluation.

9 Computational cost, memory, and history storage

9.1 Analysis work and communication

The products in Eq. 9 require approximately $O(mN_{\mathrm{e}}^2)$ arithmetic, followed by an $O(N_{\mathrm{e}}^3)$ spectral solve and $O(nN_{\mathrm{e}}^2)$ state multiplication. Streaming changes where arrays reside and how many rows are active at once; it does not remove this arithmetic. With observation chunks of at most r rows and state chunks of at most c rows, the principal analysis workspaces have dimensions

rN_e, cN_e, and N_e^2. Native local analysis has additional dependence on the number of target neighborhoods, their observation support, and the number of distinct local transforms.

The ordinary Mode-1 analysis transfers full member information to a root and then scatters state rows, while the Mode-2 global factor stage reduces three ensemble-sized products. These communication patterns have different dependencies on n, m, N_e, P, but a smaller message count does not alone imply a faster complete calculation. Mode 2 also reads and writes ensemble shards, and native streaming adds activation and store traffic. Backend geometry/inference work may require separate member or ensemble-coupled operations.

We decompose total elapsed time schematically as

$$t_{\text{run}} = t_{\text{setup}} + \sum_k \left(t_{\text{forecast},k} + t_{\text{analysis},k} + t_{\text{state I/O},k} + t_{\text{coordination},k} \right) + t_{\text{publication}}. \tag{25}$$

This is a conceptual decomposition, rather than permission to sum overlapping phase counters. In the native runner a forecast-with-analysis timer can include multiple stages; rank-summed time also differs from wall time. Verification of closed output is a further cost that we report separately. The importance of serial or fixed-overhead stages in a fixed-work scaling experiment follows the familiar argument of Amdahl (1967), while the actual balance here depends on the selected solver, process grid, and filesystem.

9.2 Memory and retention

Table 9: Memory and storage quantities that must be reported separately. Bounds describe dimensions, not measured resident memory.

Quantity	Relevant dependence or limitation
State covariance if formed	$O(n^2)$, avoided by ensemble factor analysis
Full ensemble in memory	$O(nN_e)$, not a universal property of all modes
Analysis products	$O(N_e^2)$ per product
State update block	$O(cN_e)$, configured bounded row count
Observation block	$O(rN_e)$, separate from state row bound
Persistent native states	Owned model fields times locally active members
Inactive member store	Memory or member-major HDF5 backend
PIG native noise bridge	Canonical leader workspace $O(n)$, 256 MiB allocation guard
History on disk	$O(nN_e(N_t+1))$ logical values for full retention

Native member streaming can limit active model contexts by moving inactive members to a store. Store-backed analysis is an additional option and must be selected consistently with member streaming. A memory backend and an HDF5 member-major backend have different I/O costs. Lowering active-member memory can increase file traffic; a memory claim needs measurements of both the solver and the store.

The PIG process-noise bridge uses a canonical model-group leader workspace and scatters increments to the owned rows. It avoids gathering the physical member state but still has a global-size stochastic workspace, guarded at 256 MiB. Its graph transition cache has a separate 64 MiB per-process limit. These bounded allocations are material qualifications to an otherwise distributed forecast/analysis design.

For stored item size b, a full history requires approximately

$$V_{\text{history}} = nN_e(N_t+1)b, \tag{26}$$

before means, reference trajectories, metadata, temporary publication, and duplicate export. Rolling retention removes the growth with run length for retained member states, while a complete

snapshot still represents nN_eb logical values. A VDS changes the view of backing data rather than creating another physical copy; an independently materialized export has different storage accounting (h5py developers, 2026b). We retain these distinctions when estimating capacity or moving a completed experiment.

10 The state-column time contract

10.1 Initialization is a state rather than a forecast

Let $t_0, \ldots, t_{N_t}$ denote saved model-state times. Column zero stores the physical initial state. Forecast cycle k, with $0 \le k < N_t$, advances from column k to column $k+1$. An observation attached to state column $j > 0$ is analyzed after forecast cycle $j-1$. In the implemented sequence, configured process noise is applied before that analysis.

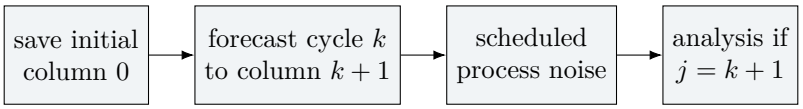


Figure 3: The corrected clock. No extra forecast is hidden in initialization. The output contains $N_t + 1$ states for N_t forecast cycles.

For the five-year validation, $\Delta t = 0.05$ years gives 100 cycles and 101 saved state columns. Observations at years 2, 3 and 4 correspond to columns 40, 60 and 80, and are applied after cycles 39, 59 and 79. The one-year short validation also included an observation at the final state. These examples distinguish the common off-by-one case of comparing a state at t_{j+1} against an observation at t_j.

10.2 Supported schedules and restart identity

`observation_cycle_steps` accepts a one-dimensional integer array of positive, strictly increasing observation columns and rejects indices past the final state. Initial-time assimilation is explicitly refused because it would require a distinct pre-forecast analysis stage. It must not be approximated by advancing once and using the old observation. The helper accepts empty schedules; that does not certify every driver configuration for forecast-only work.

`completed_observations` counts scheduled columns at or before the saved state. On restart, the next observation is the first one strictly after that state. Tests cover restarting immediately before and after analysis events so observations are neither repeated nor skipped.

Before state creation, `prepare_timestep_contract` writes `time_grid_contract.json`. The contract is `state-columns-v2`; a supplied finite, increasing time grid is hashed. The marker is published atomically and errors are broadcast. Existing histories without the contract, incompatible contract identities and changed time-grid signatures are refused. Old histories remain associated with their original convention, and a new experiment uses a distinct directory. This preserves the information needed to interpret earlier trajectories.

In Modes 0–2 and the selected instrumented paths, the analysis audit line `[ICESEE_ANALYSIS]` includes cycle, state column, model time, observation slot and observation state column. The formatter refuses a mismatch. This event log and the output verifier are more authoritative for actual completed analysis times than the older `run_schedule` summary, whose metadata helper still has a legacy terminal-index convention at this snapshot. That discrepancy is recorded as a maintenance issue, not hidden by the documentation.

11 Checkpoints, history and failure recovery

11.1 A checkpoint is a consistency boundary

A restartable checkpoint contains owned fields together with member identity, layout and schema identity, policy identity, progress counters and the model time. Native sharded checkpoints permit multiple ranks to write their owned fields. A manifest must cover every required member/variable/owned interval exactly once. Existence of a directory or a final shard is not enough to establish completeness.

A checkpoint records the completed analyzed fields and the associated replay/progress policy. Its physical interpretation also depends on when the application recomputes diagnostic quantities; the PIG native post-analysis callback synchronizes ghosts but does not itself recompute all geometry/velocity diagnostics. Completed step, next step and physical time use different conventions; inspect the recorded values rather than deriving the time from an assumed timestep. In the corrected clock, the final state after 100 forecasts is at column 100 and year 5, while the last zero-based cycle is 99.

Topology-portable restoration requires matching field ownership metadata, global IDs or supported remapping and compatible adapter capabilities. The infrastructure includes tests for remapping and checkpoint validation; a successful infrastructure test does not establish arbitrary mesh repartitioning support for every model's auxiliary solver state. The PIG-specific restart evidence must remain separate from generic application restrictions.

11.2 Logical ensemble histories and their backing files

Plotting tools expect an ensemble history with a conventional logical shape. Native state shards and mode-2 forecast files can be assembled or exposed through HDF5 virtual datasets (VDS). A VDS maps logical selections to backing datasets (h5py developers, 2026b). Its source files and relative layout remain part of a movable experiment. A metadata-only top-level HDF5 file can open successfully while its data sources are absent.

The PIG native history export uses bounded row chunks to construct the plotting history. Checkpoint retention and full-history retention are related but distinct: keeping only the last checkpoint is not enough to recreate all past members. The inspected default example retains checkpoints every cycle and enables native history export. Production storage estimates must account for these choices before submission.

When a run stops partway through, recover only the written prefix. Written-column metadata distinguish unwritten profile entries from zero-valued physical data. The recovery path can build a consistent closed view from stopped output; it must not be invoked through unsafe collectives while ranks are already failing. The reconstruction operates on closed output, after all simulation writers have stopped.

11.3 Failing collectively without hiding the cause

The full parallel driver records the original exception and uses `comm_world.Abort(1)` rather than entering unmatched cleanup collectives and returning success. Rank-local best-effort checkpoint work is different from a collective recovery export. The regression suite includes an intentional single-rank failure and a healthy negative control so a launcher that simply fails every job cannot satisfy the failure test.

We consequently distinguish three statuses: did the simulation steps complete, did the allocation complete, and did output verification complete? Each has its own status. A timeout during verification does not imply a PDE step failed; neither does a successful PDE step imply that the allocation or verification succeeded. Preserve all three statuses in the run record.

12 Verification design

12.1 Evidence tiers

Unit and contract tests establish array shape, ownership, event indexing and rejection behavior. Real-MPI tests establish communicator participation, failure termination and multi-rank I/O in the tested environment. Small integrated model runs exercise the actual driver. Backend tests require Firedrake, Icepack or ISSM and their input data. Finally, production-sized scientific validation needs the selected physical configuration and a relevant observation experiment. These tiers answer different questions.

Tests using fake communicators are useful for metadata logic but do not demonstrate that an MPI collective terminates. An executable named `mpirun` is not proof of a compatible launcher: mixed MPI installations can start disconnected singleton processes. The repository's launcher helper performs a functional two-rank probe. The functional helper and recorded rank identities establish that the test actually exercises its intended communicator.

Parallel HDF5 is another independent requirement. The standard `h5py` wheel may be serial even when `mpi4py` imports. The recorded environment therefore includes `h5py.get_config().mpi` and the actual `mpio` path when used (h5py developers, 2026a). Container and host MPI libraries must also have a supported compatibility relationship.

12.2 Comparisons must inspect members as well as means

An ensemble mean can agree while individual member errors cancel. The current Lorenz CI comparator therefore checks initial members, truth, observations, time and observation indices, all members and means pointwise through assimilation, and per-variable relative norms over the full history. An intentionally perturbed pair of members with a compensating mean is rejected by a regression test.

The norm comparison after assimilation is relevant to a chaotic free forecast, where an isolated pointwise difference near a zero crossing can be misleading. It is not permission to loosen an experiment's acceptance criteria after seeing its result. The recorded comparison retains $\mathrm{rtol} = \mathrm{atol} = 10^{-7}$, while evaluating an appropriate full-history scale after the analysis window. Report the metric, region, variables and tolerance together.

For PIG, native and generic initialization use different perturbation policies. Consequently agreement between modes 1 and 2 can be a parity experiment, but the three-mode completion experiment is not a three-mode member-by-member scientific equivalence experiment. A controlled future comparison must align initialization, noise, observation errors, filter settings, solver settings and topology.

13 Results from retained validation records

13.1 Five-year Icepack operation

The retained PACE validation used a five-year PIG configuration, 32 ensemble members and a timestep of 0.05 years. Table 10 gives the simulation-step times. The three modes ran concurrently in one two-node, 40-core allocation. Modes 1 and 2 used eight ranks each, whereas mode 3 used sixteen ranks with two per model group. These unequal resources and concurrent execution preclude a controlled speedup or scaling inference.

Each logical ensemble history has shape $(270615, 32, 101)$. Interpreted as float64, that is 874,627,680 values, approximately 6.52 GiB of logical ensemble values before metadata, backing-file overhead, additional means or checkpoints. This arithmetic is not a measured physical file size: a VDS and its backing sources have different storage accounting.

Analysis events were verified at years 2, 3 and 4. All 99 truth/member profile files across the three modes passed their reported checks. Mode-1/mode-2 final-field relative differences were

Table 10: Operational simulation results for allocation 15433074. Each simulation step exited zero. The enclosing allocation later timed out during verification.

Mode	Slurm step	MPI ranks	Ranks per model	Elapsed
1	.2	8	1	00:42:15
2	.0	8	1	01:38:31
3	.1	16	2	00:37:11

at most 9.12×10^{-13}. Native full-history thickness ranged approximately 29.94–4180.72 m and ended at 30.00–4180.68 m. Transient analyzed basal melt ranged approximately -0.3664–20.0441 in the model's configured units, ending at 0–20. These extrema establish the inspected range, not a physically accepted posterior. No undocumented clipping was added to make the range appear acceptable.

13.2 Restart and output integrity

Short allocation 15432397 completed the one-year, two-member cases and a native restart from checkpoint 13 through cycle 19, crossing two analyses. Observations were scheduled at 0.5, 0.75 and 1.0 years. The final maximum absolute restart difference was 2.99×10^{-9} and relative L2 difference 6.57×10^{-13}, within the previously declared rtol $= 10^{-9}$, atol $= 10^{-7}$. This test copied completed output before replay; it was not a demonstration of every interrupted partial-profile recovery case.

The parent five-year allocation timed out during post-run verification, after all three simulations had completed. A subsequent verification-only allocation, 15439331, also timed out while repeatedly reading the VDS. Neither allocation is reported as successfully completed. A bounded, single-CPU, read-only row-slab scan subsequently completed the remaining native field, mean, profile and manifest checks, including a short-case numerical oracle. Its reported wall time was 67.372 s, within 120 wall seconds and 2 GiB limits. No PDE work was performed by that scan.

This distinction exposed an operational bottleneck: verification access order matters for a virtual or sharded history. Reading row slabs can avoid repeatedly traversing an expensive view. Future benchmark reports should time simulation, history publication and verification separately and record the storage system, file count and cache state.

13.3 Continuous integration

The retained final CI run 42890166512 tested commit `3748e416`. It reported 754 fast tests passed, three optional skips and one deselection, followed by 42 real-MPI tests with no skips and the Lorenz integration. The Linux comparison recorded a maximum assimilation-window member difference at most 4.74×10^{-8} and a full-history per-variable relative L2 difference at most 2.15×10^{-8}.

The earlier CI attempt is retained as well: its pointwise full-history Lorenz comparison failed, despite all simulations completing. The follow-up changed the reader/comparator and regression tests rather than the scientific runtime. Keeping the failed evidence makes the reason for the comparison change reviewable. A green CI badge alone would not explain this distinction.

13.4 Checks performed for this handover

The exact inspected source was also tested locally during documentation preparation. A focused set of 184 tests passed with no skips, covering clock, ownership, streamed analysis, native state and recovery contracts. Twelve real local MPI tests passed with no skips; these included synthetic one-rank checkpoint write failure, changed-process-grid restoration, MPI abort followed

by partial-history export, topology and healthy/failing controls. Synthetic failure workers do not replace a deliberately interrupted full ice-sheet model experiment.

The existing Lorenz CI integration was rerun locally in modes 0, 1 and 2 with its unchanged 30-member, 1,000-cycle configuration and isolated output paths. Each history had shape $(3, 30, 1001)$ and ten observation slots. The mode-1/mode-2 maximum member difference through assimilation was 2.34×10^{-8}; full-history per-variable relative L2 was at most 3.62×10^{-9}. The all-time maximum absolute difference was 2.37×10^{-7}, so the accepted full-history result must not be represented as an all-time pointwise 10^{-7} bound. The current repository comparator accepted the histories using its specified window/norm criteria.

The local environment used Python 3.11.12, NumPy 2.2.6, h5py 3.13.0 with MPI support and Open MPI 5.0.9. The first focused invocation encountered three inherited scratch-directory permission failures; task-private writable scratch resolved them without source changes. Both invocations and their environment scope are retained. No new PACE allocation or container build was performed; the later coarse Icepack exercise is reported in Section 14.

13.5 A terminal-observation integration comparison

We also run a targeted local fixture with two MPI ranks, eight members, and 50 forecast cycles. Observations occur at model times 0.2, 0.4, 0.6, 0.8, and 1.0, including the terminal state. Modes 1 and 2 produce finite member histories of shape $(3, 8, 51)$, means consistent with those members, and five matching audit events at state columns 10, 20, 30, 40, and 50. Their maximum absolute complete-history difference is 5.31×10^{-9}, and the final-state maximum difference is 9.10×10^{-10}.

The same fixture completes in the Lorenz Mode-3 runner with five reported analyses and complete finite output. That runner does not emit the same event-line format, and its realized native prior/policy is not used as a member-parity reference here. We therefore report its completion separately. The fixture and verification record are retained under `paper/production-report/experiments/`; the result specifically tests terminal state indexing and the selected small-model paths, rather than distributed glacier physics or a performance ranking.

13.6 Scope of the results

The records do not establish a new strong- or weak-scaling curve, a universal resident-memory bound, posterior calibration, improved real-data skill, or equivalence across all registered native adapters. They do not establish reproducibility of every container image or of an arbitrary Spack concretization. They also do not establish a physical acceptance tolerance from the observed differences. These are separate experiments with explicit controls, as specified in Section 15.

14 Reproducible sample applications

We provide progressively more model-specific exercises so students can connect the algorithmic description to an inspectable run. Each uses a dedicated fresh output directory, a retained configuration, the actual application entrypoint, and closed-output verification. The small cases are learning and regression experiments; their reduced meshes, priors, and uncertainty amplitudes are not calibrated scientific recommendations.

The Lorenz endpoint fixture uses eight members, 50 forecasts, and observations at five times including the terminal state. The two-rank Mode-1/Mode-2 comparison and Mode-3 completion results are reported above. Figure 4 shows actual retained Mode-1 synthetic truth, background, observations, member trajectories, and mean for the three components. The CSV and plotting script reproduce the vector figure. This visual trace shows the complete saved calculation and is not a controlled filtering-skill or mode-speedup study.

The new coarse Icepack exercise is a configured case under `synthetic_ice_stream/tutorials/coarse_assimilation`. Its wrapper delegates the existing synthetic-stream driver and

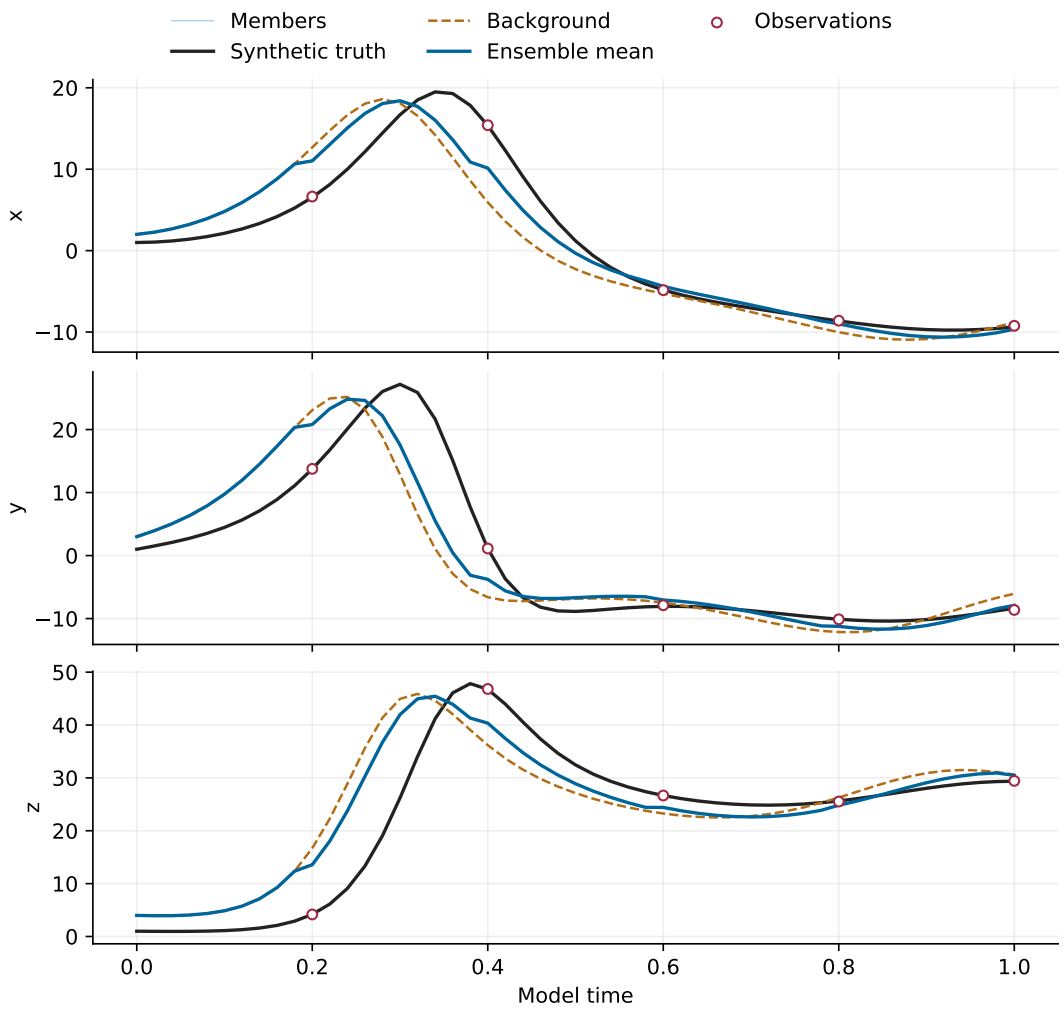


Figure 4: Retained eight-member Mode-1 Lorenz fixture with 50 forecasts and five observation times, including the final state. The historical directory is named `lorenz96`, while these are the three-variable Lorenz-63 equations. Curves show synthetic truth, background, all members and their mean; symbols show the generated observations. Numerical CSV and a Matplotlib script accompany the report.

hooks instead of copying a physical solver or pretending to register a new native model. The generated rectangular mesh needs no external mesh/spin-up dataset. The child YAML specifies 6×4 quadrilateral elements, degree-one fields, four members, one year, and ten forecasts, with observations at 0.5 and 1.0 year. For one spatial rank the four blocks $h, u, v,$ smb give an expected history shape $(140, 4, 11)$. It explicitly clears the inherited restart-position override, disables localization and the optional inference plugin for the first exercise, and fixes one model rank per member. Mode 0 runs in one Python process; Modes 1–3 use two MPI ranks for the retained exercise, with one model rank per member. All four completed ten forecasts and two analyses, including the final observation. Closed-output checks found finite values and positive thickness. Generic Modes 0–2 produced the expected $(140, 4, 11)$ histories with means consistent with the members. Mode 3 wrote eleven native state files of shape $(140, 4)$ and a sharded history, without the generic ensemble filename or a separately stored mean. The Mode-1/Mode-2 maximum pointwise difference was 2.0078×10^{-10}, below the retained absolute comparison tolerance of 10^{-7}. The machine-readable record `experiments/icepack-coarse-results.json` retains the topology and output contracts. These results exercise the actual Firedrake/Icepack model; the coarse case remains a workflow check rather than a spatial-convergence or posterior-skill experiment.

The short ISSM exercises under `ISMIP_Choi/tutorials/short_assimilation` and `basal_friction_variation/tutorials/short_assimilation` retain their own reference-model and MATLAB/ISSM requirements. They provide four-member, two-year profiles for fixed, EnKF-updated, or hybrid friction treatment through existing drivers. The Choi case requires its approved bed prior and remains distinct from the executed basal-friction case.

The capped basal hybrid calculation used one Python rank in Mode 1 and one internal ISSM process per solve. It produced a finite $(20082, 4, 11)$ ensemble with consistent means and analyses at years one and two, mapped to columns five and ten. At year two, all four members invoked the inverse; each returned 3,347 finite friction coefficients that matched the published final ensemble exactly. Every optimizer stopped on `dxmin` during line search. We therefore report completed inverse execution and verified field handoff, rather than gradient-tolerance convergence, objective improvement or calibrated parameter skill. The printed final trial values need not be accepted objective values. The record `issm-basal-results.json` retains the distinct basal objective, bounds, diagnostics, source/input hashes and fixed/EnKF/hybrid statuses.

Preparing this exercise exposed three legacy lifecycle defects: an HDF5 callback returned from the ISSM working directory, reference output shifted the first forecast into column zero and omitted the terminal state, and an early deferred inverse lost its requested switch. The later corrections restore the application directory, export the actual pre-solve state at column zero with matching subsequent columns, and preserve the requested inverse across deferred events. Existing transient solves and the one-year prior spin-up are retained. The profiles use the existing resource-retention switch to keep the requested internal process count; uncapped attempts and their effective resources remain separate evidence. These corrections and the Choi missing-result guard have focused regressions. None of the basal results validates the source-inspected Choi geometry/inversion handoff or its native restart capabilities.

The companion sample-runs guide lists the exact commands and current execution evidence. A new application can use these cases as a template for configuration, registration, and verification while retaining the state, external-process, and inverse-handoff distinctions developed in Sections 5 and 6. In particular, student reproducibility requires the complete parent/child configuration chain, numerical environment, model inputs, launch topology, output checks, and the scientific meaning of each selected inference stage.

15 Discussion and directions for scientific evaluation

15.1 What the execution comparisons establish

The developments separate several conditions that previously could be conflated: an allocated process, an ensemble member, an owned physical field, a completed state, and a published history. This separation lets us ask a precise question when changing a representation. For Modes 1 and 2, the relevant numerical test is whether the same realized forecast and observation experiment yields equivalent complete members. For native fields, an additional question is whether packed owned rows refer to the same physical degrees of freedom and whether model consistency is restored after analysis.

The retained PIG calculation demonstrates that the inspected configurations can complete their simulation steps and produce histories that pass the recorded integrity checks. The short replay demonstrates agreement across selected checkpoint/analysis boundaries. The additional final-time fixture exercises the terminal state explicitly. These are complementary results: a lightweight driver can isolate event and communication behavior, whereas a finite-element run exercises real model state and output. Neither alone establishes posterior calibration or real-data skill. The whole-member native bindings also remain different scalability cases from the spatial PIG interface.

The exact finite-sample factor is an important methodological boundary. We derive the algebra used by the implementation and preserve its selected perturbation policy across streamed and ordinary representations. Its cross terms and spectral truncation warrant a controlled comparison against a clearly specified covariance formulation if a new filtering-method claim is intended. Such a study would vary ensemble size, observation structure, and localization while holding initialization and model dynamics fixed. It would complement the present engineering equivalence tests rather than being inferred from them.

15.2 Controlled scaling and memory experiments

For strong scaling, we would hold the mesh, ensemble, number of cycles, observations, noise realizations, solver settings, history policy, and numerical stack fixed, while varying a declared process-grid dimension. Repeated runs would report variability and separate forecasts, global/local analysis, state traffic, and final publication. Measurements of resident memory per rank, total memory, file volume, and inactive-member traffic would show whether a reduced active-model footprint is offset by another workspace or I/O stage.

For weak scaling, the amount of work that grows with the process count must be defined. Scaling member count differs from scaling spatial degrees of freedom, and both differ from scaling the observation density or number of local neighborhoods. Constant local state size does not imply constant solver iterations or analysis support. The concurrent, unequal-resource timings in the present record therefore remain operational measurements, while a future scaling experiment would isolate these factors.

15.3 Physical experiments and reproducible use

We would evaluate scientific performance first with a controlled twin experiment and explicit synthetic observation errors, then with independent or withheld observations for any real-data skill claim. Forecast-only and assimilated trajectories should be compared through state error, spread, coverage, innovations, geometry, and parameter plausibility. The ice-sheet assimilation studies reviewed earlier show why observation design and uncertain basal conditions are relevant questions (Bonan et al., 2014; Gillet-Chaulet, 2020; Choi et al., 2025); they do not determine an acceptance threshold for the PIG configuration reported here.

The transient basal-melt extrema in the retained native history illustrate the distinction. Their finite values establish an inspected numerical range, while interpretation of temporarily

negative melt or values above a configured nominal range depends on the experiment's sign convention and physical assumptions. A projection or bound would be a declared change to the numerical/scientific policy, followed by a new comparison. We retain the observed range so that this decision remains scientifically visible.

Finally, reproducible use requires access to exact inputs, source, configuration, and environment as well as output. The documentation and appendices provide procedures for student-led operation, including source selection, coherent MPI/HDF5 environments, fresh-output isolation, and accepted restart boundaries. Publication requires a persistent archive of this later snapshot and its supporting experiment, as reflected in GMD's code/data policy (Geoscientific Model Development, 2026). A JAMES-oriented article would additionally benefit from the controlled Earth-system numerical or physical result described above. Venue choice and submission follow those completed scientific and archival decisions.

16 Conclusions

We describe parallel ensemble data assimilation in ICESEE through the operations that connect a forecast model, an ensemble analysis, and a recoverable experiment. The four execution modes use different distributions of members, state rows, observation products, and model objects. Mode 0 provides a serial lifecycle with method-specific analysis costs; Mode 1 combines member forecasts with a root transform and row updates; Mode 2 streams file-backed ensemble analysis; and Mode 3 exposes application-specific native execution, including spatial PIG fields and separately identified whole-member bindings.

Hybrid inversion adds a separately weighted and bounded model-control problem to the ensemble update. Its activation, geometry, return-field and failure-publication policies determine what parameter is actually passed to the next forecast. Optional SMB and bed operations instead regularize apparent balance or proposed bed increments; their bases, support, ordering and persistence remain part of the inference method.

The implemented factor transform can be evaluated from ensemble-sized products while preserving its stated finite-sample construction. Explicit ownership prevents replicated copies or ghosts from becoming additional state, and the state-column convention connects initialization, forecast cycles, observation events, and checkpoint progress. Persistent/native and store-backed policies address different parts of the memory requirement, while sharded checkpoints and bounded history export separate restart state from a retrospective ensemble history.

The retained simulations, restart checks, CI results, and additional small MPI experiments support the reported configurations and numerical comparisons. They also identify the next questions: controlled scaling, active and auxiliary memory costs, filtering-factor assessment, and physical uncertainty/skill evaluation. By making these boundaries explicit and providing a detailed technical reading route, we support continued development and reproducible student-led ice-sheet experiments without treating a completed job as the complete scientific result.

Code and data availability

The implementation is available from `https://github.com/ICESEE-project/ICESEE` (ICESEE project, 2026), with this report tied to commit `3748e416d3a8120cc860119bd7224c6f9ad41ff7`. Installation wrappers are available at `https://github.com/ICESEE-project/ICESEE-Spack` and `https://github.com/ICESEE-project/ICESEE-Containers`, with inspected revisions in Table 11. The top-level source license text is BSD 2-Clause; conflicting package metadata are identified in Appendix A. The companion evidence package retains verification JSON, selected logs, source hashes and a machine-readable registry. Full PIG inputs and ensemble data from the recorded PACE runs have not been newly deposited with this draft. A persistent archive containing the precise new source, inputs or their access arrangements, configuration, launch

scripts, outputs and figure-generation code is required before journal submission. The prior paper's archive must not be cited as the archive of this later snapshot.

Author contributions and acknowledgements

This manuscript is a draft attributed to Brian Kyanjo. Final author participation, contribution statements, affiliations, and funding acknowledgements will reflect the contributions to the completed study before public submission. The retained numerical experiments used Georgia Tech PACE resources. The documentation and local validation described here did not submit an additional production allocation.

A Source identity and the computational experiment

A.1 A source revision is part of the experiment

The code basis is the published development branch at the hash shown in Table 11. At the time of inspection, `main` and `features_debug` differed. A fresh clone of the default branch therefore cannot be assumed to contain the implementation described here. Pull request 72 remained open in the retained publication record. Documentation work does not merge that pull request or update an installation's parent repository pin.

Table 11: Source identities inspected for this report. Long hashes, rather than moving branch names, identify the snapshots.

Component	Inspected identity
ICESEE development source	`3748e416d3a8120cc860119bd7224c6f9ad41ff7`
ICESEE default branch	`3d7e6e757e07b80dcca1fdd65dffb8ca8853316a`
ICESEE-Spack main	`68e37fffcd13cc60636028d051d4637e7f573d4f`
ICESEE-Containers main	`884cc3046a26b5999f6db16a98ad38a96bfdbcfd`
Wiki before this documentation update	`79a45cc643f2a2943e10e8d8d1dccf99253be69e`

The small basal-friction runs additionally use the later runtime corrections through commit `02ac549`; their exact full source identities and hashes are retained in `issm-basal-results.json`. The core ensemble formulation remains identified by 3748e416d3a8120cc860119bd7224c6f9ad41ff7.

The repository's metadata are not internally consistent at this snapshot: `pyproject.toml` declares version 0.1.9 and an MIT license string; `CITATION.cff` declares 1.0.0; the top-level license text is BSD 2-Clause. The original paper and the citation file also refer to different archive identifiers. The report therefore uses a commit identity, cites the existing paper as prior work, and does not represent an older archive as containing these developments. Resolving release metadata and depositing the exact new source and data are required before submission.

The execution environment is equally important. A source checkout, a Python editable installation, a Spack environment and a container image are distinct objects. The interpreter can import an older source path even after a newer checkout is fetched. Before running, record the executable, `ICESEE.__file__`, the package root, the Git hash, local changes, the MPI library reported by `mpi4py`, and HDF5 parallel support. These records must be generated within the actual batch environment.

A.2 What must accompany a result

An experiment consists of source, configuration, inputs, numerical environment, launch topology, random policy, output and validation. Save the resolved configuration as well as the original

YAML and CLI arguments. Preserve input checksums or persistent dataset identifiers. Record the number of world ranks, ranks in each model group, thread limits and node placement. Save simulation exit status separately from the status of a postprocessing or verification command.

A branch name alone is insufficient because it can move; a container tag alone is insufficient for the same reason. A YAML file alone is insufficient because command-line options can replace its values. A final image is insufficient because it can hide missing members or an incorrect observation schedule. These records are small relative to the ensemble history and allow another operator to reconstruct the intended calculation.

B Installation and environment operation

B.1 Choose one coherent numerical stack

Use an existing verified numerical stack when updating Python source. Rebuilding Firedrake, PETSc or MPI is not a required consequence of a source documentation update. For a new environment, select a Spack installation or a container path, retain the exact wrapper revision, and pass the same environment probes used for a native installation. Do not combine libraries from multiple installations simply because each imports separately.

The current ICESEE-Spack parent uses Git submodules. Its inspected bundled ICESEE pin is `a793cd198ab39eedc668f3f67e08d56ba9bf22dc`, which differs from this report's source. A parent update, a child fetch and a dependency rebuild are distinct actions. Inspect `gitsub modulestatus` and the child's origin and hash before selecting a development revision. Older instructions in the workspace describe a subtree-based wrapper and must not be applied to current main.

Activation is executable policy. The current wrapper can purge modules, load MATLAB, preload a compiler runtime and prepend bundled ICESEE source paths. Record module state before and after activation and check the actual import. The wrapper's convenience test script contains exception-catching behavior and a name mismatch in an ICESEE print statement; its exit status alone is not a trustworthy installation gate. Use direct imports, functional MPI and HDF5 probes and a small integrated run.

B.2 Containers and HPC launch boundaries

A container provides a filesystem and libraries; it does not automatically provide the cluster's supported MPI fabric or scheduler integration. Record the image digest or SIF checksum, build recipe revision, bound host directories and MPI strategy. Use the checked-in recipe files and wrappers whose actual paths are listed in the deployment guide rather than assuming a Docker launch is directly suitable on a compute node.

At the inspected container revision, combined wrappers select Open MPI for Firedrake/Icepack/ICESEE and MPICH for ISSM. A wrapper therefore selects a numerical environment as well as an application. Several recipes bootstrap mutable image tags or clone moving source branches, so pinning a recipe alone does not freeze its result. A separately pinned ICESEE checkout can be bound to `/opt/ICESEE` for the combined wrappers; verify the imported location and retain the image digest or SIF checksum. A source bind does not update compiled dependencies. Host/container MPI launches were not newly validated in this documentation work.

Inputs and outputs must be visible inside every launched rank. Bind the full run directory, reference data and any VDS backing sources. Keep writes in an allocated project/scratch directory with adequate quota, not an immutable image layer. Run compiler, PyOP2 and SQLite caches on node-local scratch with job/step/rank isolation where supported by the launcher. A shared cache can create locks or mix stale build state across processes.

The companion deployment guide provides the pinned repository commands and explicitly marks paths requiring site-local values. Those commands are installation procedures, not evidence that the corresponding image or environment has been newly built and benchmarked here.

C A production operations protocol

C.1 Before the first production run

Assign a primary operator and a backup operator from the available student team. Assign a scientific reviewer for experiment changes and an infrastructure contact for scheduler or filesystem issues. These are roles; no individual is presumed to have accepted responsibility. Store the role assignment with the project run register so absence of the original developer does not leave an ambiguous decision path.

Begin with a clone or a known clean checkout of the pinned source. Inspect local changes before pulling or switching branches. Do not reset another operator's work. Verify the environment in the batch context, then run the small driver and relevant MPI/backend checks in isolated temporary output directories. Save the resolved configuration and input manifest before a production launch.

Fresh-run configuration loading can recursively remove an existing `data_path`. Consequently a unique run directory is a correctness requirement, not merely a naming preference. The `restar t_enabled` setting alone is not a cleanup exemption; the explicit `resume_from_checkpoint` path must be used as documented for the chosen adapter. Do not import application configuration merely to inspect its defaults in a directory containing valuable output.

C.2 During a run

Monitor scheduler state and elapsed time, simulation-step exits and the model's own progress. A quiet solver is not necessarily hung, and a printed progress line is not proof of collective progress. Diagnose using process state, checkpoint timestamps, completed-state counters and the last analysis audit event. Preserve stdout and stderr from every relevant step.

Do not edit YAML or source in place during an active run and assume the process will adopt it consistently. Record a new run identity for any change of timestep, observation schedule, solver settings, process-noise policy, initialization, inflation or localization. Size storage for checkpoints and backing history files before a long allocation begins.

C.3 After a run

Treat the output as unaccepted until the simulation status, expected final time, member/variable coverage, finite values, mean consistency, observation events, profiles and checkpoint manifest pass. Verify the data in bounded slabs when the storage view is expensive. Preserve any timeout status even if verification later succeeds elsewhere.

For a stopped run, first confirm that no process still writes the directory. Copy or snapshot the recovery inputs, inspect the latest complete checkpoint and the time-grid marker, and use the adapter's documented restore path. Rebuild a partial history only from a verified written prefix. Never delete the original history to create space for its recovery view.

C.4 Six months of unattended developer absence

Operate a small number of pinned, approved experiment templates. Operators may resubmit an unchanged configuration after a scheduler interruption using the documented restart path, provided the checkpoint and environment gates pass. Scientific settings, filter selection, initialization policy or adapter code changes require review by the designated scientific/developer role. When review is unavailable, preserve evidence and continue unaffected approved work.

Use a run register containing identity, operator, purpose, source, environment, input manifest, launch command, allocation and step IDs, output location, acceptance status and recovery history. Review storage growth and a representative restore periodically as part of normal project operations. Before upgrading the stack, reproduce a small accepted baseline in a separate environment and compare members, means and observations. Retain the old environment until the replacement passes.

An incident record should answer what was intended, what happened, which state was last complete, which data were preserved, what hypothesis was tested and what decision was made. This turns debugging into accumulated project knowledge rather than an oral dependency on the absent developer.

D Source map for a new developer

Table 12: Where to read the implementation. Paths are relative to the pinned ICESEE root.

Path	Responsibility and reading question
`config/_utility_imports.py`	Configuration import and CLI handoff; examine cleanup behavior before importing.
`applications/supported_models.py`	Model registry and adapter/module resolution.
`src/run_model_da/`	Execution dispatch and mode-specific orchestration.
`src/utils/state_ownership.py`	Replicated versus distributed dimensions and combination rules.
`src/utils/timestep_contract.py`	Saved-state clock, observation cycle mapping and compatibility marker.
`src/utils/run_schedule.py`	Run metadata helper; note terminal-index convention limitation.
`src/parallelization/distributed_topology.py`	Spatial and ensemble communicators and membership.
`src/parallelization/distributed_adapter.py`	Owned slab metadata and segmented block layout.
`src/parallelization/distributed_fields.py`	Native field layout and ownership registry.
`src/parallelization/distributed_analysis.py`	Streamed products and spectral ensemble transform.
`src/parallelization/distributed_local_analysis.py`	Grouped/local targets and observation support.
`src/parallelization/distributed_native_runtime.py`	Persistent member contexts and owned snapshots.
`src/parallelization/distributed_native_cycle.py`	Global/local analysis cycle, inversion and restore orchestration.
`src/parallelization/distributed_member_store*.py`	Inactive member persistence backends.
`src/parallelization/distributed_checkpoint.py`	Sharded field checkpoints and metadata validation.
`src/parallelization/distributed_history_export.py`	Bounded construction of logical output histories.
`src/parallelization/mode3_checkpointing.py`	Application-facing native checkpoint/history policy.
`src/parallelization/EnKF_parallel_io.py`	Generic file-backed ensemble I/O and finalization.
`src/parallelization/native_process_noise.py`	Canonical stochastic workspace and owned increments.
`src/parallelization/native_application.py`	Generic native application capabilities and refusals.

Path	Responsibility and reading question
`applications/icepack_model/icepack_utils/`	Native Firedrake field and communicator bridge.
`applications/icepack_model/examples/idealized_pig/`	Physics, configuration, native runner, batch launcher and output verifier.
`scripts/ci/run_lorenz96_ci.py`	Integrated tiny-system driver and member/history comparator.
`src/tests/`	Contract tests, real MPI tests and optional backend checks.

E Evidence inventory

The accompanying evidence registry maps each measured claim to retained source records and their SHA256 hashes. Its current records include `full-15433074-verification.json`, `full-15433074-topology.json`, `full-15433074-comparison.json`, `short-15432397-verification.json`, `ci-final.json`, `RESULTS.txt` and the publication logs. The intermediate `CURRENT_RESULT.json` is preserved but superseded by the completed verification and publication records. Earlier-clock cases are explicitly historical.

New local records are `focused-tests-result.json`, `mpi-tests-result.json` and `lorenz-integration-result.json`, together with command logs and JUnit output. The registry's audit verifies copied hashes and numerical cross-checks; it does not fetch live state or repeat simulations. Selected verification records are distributed with the manuscript source. Full PACE raw histories and the complete input bundle remain a separate archive requirement.

F The minimum run record

The following record is a specification, not an assertion that fields have been populated for an unperformed run. Keep it next to the resolved configuration and validation report.

```
run_id: unique immutable identifier
purpose: hypothesis or approved production template
operator: primary and backup roles
source: remote URL, commit, local patch hash, actual import path
environment: interpreter, dependency lock, MPI, PETSc, HDF5
deployment: Spack parent/child commits or image digest
inputs: paths, checksums, provenance and access arrangements
science: resolved YAML, CLI, initialization and random policy
topology: nodes, world ranks, model groups, ranks per model
execution: exact command, job ID, step IDs, stdout/stderr
output: unique path, time-grid contract, checkpoint/history policy
simulation_status: exit and final physical model time per step
allocation_status: scheduler state and exit
verification_status: checks, tolerances, missing evidence
recovery: checkpoint identity, prior run and validation result
acceptance: reviewer role, decision, date and known limitations
```

G Failure diagnosis and decisions

Table 13: A practical diagnosis sequence. Preserve the failed run before any retry.

Observed symptom	First evidence to inspect	Appropriate next action
Import selects old code	Executable, `ICESEE.__file__`, activation script and paths	Fix environment selection in a separate shell; repeat provenance probes.
MPI appears to use one rank repeatedly	Functional launcher probe, rank/world size and MPI library	Select compatible launcher; do not accept a singleton run as a parallel test.
HDF5 `mpio` unavailable	`h5py.get_config().mpi`, linked libraries	Use matching parallel HDF5 build for that path.
Output disappears on startup	Requested data path, fresh/resume flags and cleanup import	Stop; preserve remaining files; use a fresh path or supported explicit resume.
Analysis refused for collapsed spread	Initial member spread and process-noise events	Diagnose scientific configuration; do not suppress the guard.
Old checkpoint rejected	Clock signature, schema, policy and source identities	Preserve old experiment; rerun fresh or use its compatible source.
Allocation timeout after final simulation	Step exits, final checkpoint time, verifier position	Record timeout; verify closed output separately in bounded reads.
VDS opens but values absent	Source paths, manifests and retained backing files	Restore complete backing layout; never assume top-level file is self-contained.
Mean agrees but members differ	Member-wise fields and analysis-window comparisons	Reject mean-only parity; isolate identity/order/random policy.
Posterior finite but implausible	Constraints, units, innovations and field ranges	Obtain scientific review; finite arithmetic is not physical validation.

H A worked first-run procedure

Activate the selected numerical environment before the following commands. Run them from a documentation checkout named `ICESEE` containing the companion handbook and its complete smoke YAML. Confirm the printed import points to that checkout. The procedure uses a new temporary directory, not a production output directory.

```
export ICESEE_ROOT="$(pwd -P)"
export PYTHONPATH="$(dirname "$ICESEE_ROOT"):$ICESEE_ROOT${PYTHONPATH:+:$PYTHONPATH}"
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1
git rev-parse HEAD
git status --short
python -c 'import ICESEE,sys; print(sys.executable); print(ICESEE.__file__)'
CASE_ROOT="$(mktemp -d "${TMPDIR:-/tmp}/icesee-student.XXXXXX")"
cp docs/handbook/examples/lorenz-smoke.yaml "$CASE_ROOT/params.yaml"
python applications/lorenz_model/examples/lorenz96/run_da_lorenz96.py \
  -F "$CASE_ROOT/params.yaml" --execution_mode=0 \
  --data_path="$CASE_ROOT/mode0" \
  > "$CASE_ROOT/stdout.log" 2> "$CASE_ROOT/stderr.log"
```

The supplied configuration specifies eight members, 50 cycles and observations at 0.2, 0.4, 0.6 and 0.8 model-time units. The observed serial smoke test completed with member shape $(3, 8, 51)$, mean shape $(3, 51)$, finite values and a mean/member consistency difference of approximately 7.11×10^{-15}. State columns 10, 20, 30 and 40 match the four observation times. Inspect the closed HDF5 datasets and logs before accepting a new execution; these shapes are expectations for this fixture rather than arbitrary applications.

Before attempting a multi-rank mode, run the repository's functional launcher probe and check parallel HDF5 as described in the handbook. For PIG, read the example's required input list and environment resolver, then use a new `ICESEE_DATA_PATH` with its batch launcher only inside the group's allocation policy. The small serial fixture does not establish that the Icepack environment, inputs or recovery path are ready.

I A student learning sequence

First trace a single observation from configured model time to saved state column, forecast cycle and audit event. Reproduce the final-time observation unit test and explain why column zero is refused. Next build a tiny observation ensemble and verify that one-chunk and multiple-chunk accumulation give the same transform within floating-point tolerance. State the dimensions of every product and explain why the two pseudoinverse factors occur.

Then inspect the small Lorenz driver. Write down its actual equations and compare mode-1/mode-2 members through assimilation, not just the means. Perturb two members by equal and opposite increments to demonstrate why the mean is insufficient. Read the ownership tests and explain why replicated ranks must not be concatenated.

Finally inspect the PIG launcher's environment, inputs, unique output path, timestep, ensemble size, checkpoint policy and verifier. With approved resources, complete a short run and replay a checkpoint across an analysis. Compare profiles and distinguish simulation, allocation and verification status.

References

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. Proceedings of the April 18-20, 1967, Spring Joint Computer Conference, 483–485. https://doi.org/10.1145/1465482.1465560.

Balay, S., Abhyankar, S., Adams, M. F., et al. (2025). PETSc/TAO Users Manual. Argonne National Laboratory, ANL-21/39 - Revision 3.23. https://doi.org/10.2172/2565610.

Bonan, B., Nodet, M., Ritz, C., and Peyaud, V. (2014). An ETKF approach for initial state and parameter estimation in ice sheet modelling. Nonlinear Processes in Geophysics, 21(2), 569–582. https://doi.org/10.5194/npg-21-569-2014.

Burgers, G., van Leeuwen, P. J., and Evensen, G. (1998). Analysis Scheme in the Ensemble Kalman Filter. Monthly Weather Review, 126(6), 1719–1724. https://doi.org/10.1175/1520-0493%281998%29126%3C1719%3AASITEK%3E2.0.CO%3B2.

Carrassi, A., Bocquet, M., Bertino, L., and Evensen, G. (2018). Data assimilation in the geosciences: An overview of methods, issues, and perspectives. WIREs Climate Change, 9(5), e535. https://doi.org/10.1002/wcc.535.

Choi, Y., Petty, A., Felikson, D., and Poterjoy, J. (2025). Estimation of the state and parameters in ice sheet model using an ensemble Kalman filter and Observing System Simulation Experiments. The Cryosphere, 19, 5423–5444. https://doi.org/10.5194/tc-19-5423-2025.

Evensen, G. (2003). The Ensemble Kalman Filter: theoretical formulation and practical implementation. Ocean Dynamics, 53, 343–367. https://doi.org/10.1007/s10236-003-0036-9.

Gaspari, G., and Cohn, S. E. (1999). Construction of correlation functions in two and three dimensions. Quarterly Journal of the Royal Meteorological Society, 125(554), 723–757. https://doi.org/10.1002/qj.49712555417.

Geoscientific Model Development (2026). Model code and data policy. Accessed 10 November 2026. https://www.geoscientific-model-development.net/policies/code_and_data_policy.html.

Gillet-Chaulet, F. (2020). Assimilation of surface observations in a transient marine ice sheet model using an ensemble Kalman filter. The Cryosphere, 14(3), 811–832. https://doi.org/10.5194/tc-14-811-2020.

h5py developers (2026a). Parallel HDF5. h5py 3.15 documentation. Accessed 10 November 2026. https://docs.h5py.org/en/3.15.0/mpi.html.

h5py developers (2026b). Virtual Datasets (VDS). h5py 3.15 documentation. Accessed 10 November 2026. https://docs.h5py.org/en/3.15.0/vds.html.

The HDF Group (2026). Collective Calling Requirements in Parallel HDF5 Applications. HDF5 documentation. Accessed 10 November 2026. https://support.hdfgroup.org/documentation/hdf5/latest/collective_calls.html.

Hunt, B. R., Kostelich, E. J., and Szunyogh, I. (2007). Efficient data assimilation for spatiotemporal chaos: A local ensemble transform Kalman filter. Physica D: Nonlinear Phenomena, 230(1-2), 112–126. https://doi.org/10.1016/j.physd.2006.11.008.

ICESEE project (2026). ICESEE source, development snapshot 3748e416d3a8120cc860119bd7224c6f9ad41ff7. https://github.com/ICESEE-project/ICESEE/tree/3748e416d3a8120cc860119bd7224c6f9ad41ff7.

Kyanjo, B., Mayo, T. L., and Robel, A. A. (2026). The Ice Sheet State and Parameter Estimator (ICESEE) Library (v1.0.0): Ensemble Kalman Filtering for Ice Sheet Models. arXiv, 2603.26947v1. https://doi.org/10.48550/arXiv.2603.26947.

Kyanjo, B., Mayo, T. L., and Robel, A. A. (2026b). Public discussion and author replies for egusphere-2026-2037, including August 2026 replies on observation-error modes and grouped local analysis. https://egusphere.copernicus.org/preprints/2026/egusphere-2026-2037/.

Larour, E., Seroussi, H., Morlighem, M., and Rignot, E. (2012). Continental scale, high order, high spatial resolution, ice sheet modeling using the Ice Sheet System Model (ISSM). Journal of Geophysical Research: Earth Surface, 117(F1), F01022. https://doi.org/10.1029/2011JF002140.

MacAyeal, D. R. (1993). A tutorial on the use of control methods in ice-sheet modeling. *Journal of Glaciology*, 39(131), 91–98. https://doi.org/10.3189/S0022143000015744.

Morlighem, M., Rignot, E., Seroussi, H., Larour, E., Ben Dhia, H., and Aubry, D. (2010). Spatial patterns of basal drag inferred using control methods from a full-Stokes and simpler models for Pine Island Glacier, West Antarctica. *Geophysical Research Letters*, 37(14), L14502. https://doi.org/10.1029/2010GL043853.

Morlighem, M., Seroussi, H., Larour, E., and Rignot, E. (2013). Inversion of basal friction in Antarctica using exact and incomplete adjoints of a higher-order model. *Journal of Geophysical Research: Earth Surface*, 118(3), 1746–1753. https://doi.org/10.1002/jgrf.20125.

Message Passing Interface Forum (2025). MPI: A Message-Passing Interface Standard, Version 5.0. Message Passing Interface Forum, Version 5.0. https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf.

Rathgeber, F., Ham, D. A., Mitchell, L., Lange, M., Luporini, F., McRae, A. T. T., Bercea, G.-T., Markall, G. R., and Kelly, P. H. J. (2016). Firedrake: Automating the Finite Element Method by Composing Abstractions. ACM Transactions on Mathematical Software, 43(3), 24:1–24:27. https://doi.org/10.1145/2998441.

Sakov, P., and Oke, P. R. (2008). A deterministic formulation of the ensemble Kalman filter: an alternative to ensemble square root filters. Tellus A: Dynamic Meteorology and Oceanography, 60(2), 361–371. https://doi.org/10.1111/j.1600-0870.2007.00299.x.

Shapero, D. R., Badgeley, J. A., Hoffman, A. O., and Joughin, I. R. (2021). icepack: a new glacier flow modeling package in Python, version 1.0. Geoscientific Model Development, 14(7), 4593–4616. https://doi.org/10.5194/gmd-14-4593-2021.

Tippett, M. K., Anderson, J. L., Bishop, C. H., Hamill, T. M., and Whitaker, J. S. (2003). Ensemble Square Root Filters. Monthly Weather Review, 131(7), 1485–1490. https://doi.org/10.1175/1520-0493%282003%29131%3C1485%3AESRF%3E2.0.CO%3B2.